

实证研究 Agent ·设计文档集

SPEC v0.5 ·详细设计 DD-01—07 ·Stata 实证审计

Stata Agent 项目

2026-09-07

目录

实证研究 Agent ——项目交接总览 (v0.5)	1
0. 一句话定位	1
1. 已钉死的决策 (多轮访谈 + v0.4 修订, 见 SPEC §1)	1
2. 面试/工业级四维 (已定设计, 见 SPEC §4.9)	2
3. 技术选型 (v0.4, 见 SPEC §1.5)	3
4. 架构分层 (见 SPEC §3)	3
5. 一个 idea 的工作流 (见 SPEC §2, 跑在阶段状态机上)	4
6. 路线 (给 codex/新 session 的切法, v0.4 合并 P1/P2, 见 SPEC §5)	4
7. 待深化/待试 (见 SPEC §7 与 research 提示词)	4
8. 文件导航	4
实证研究 Agent ——设计规格 (v0.5, 工业级)	7
修订记录	7
0. 一句话定位	8
1. 已钉死的决策 (多轮访谈 + v0.4/v0.5 修订)	8
2. 工作流 (一个 idea 的生命周期, 跑在阶段状态机上)	12
3. 系统架构	13
4. 关键模块与数据契约	13
5. 分阶段路线 (v0.4 合并 P1/P2; v0.5 并入 C 项)	20
6. 给 codex 的默认建议 (可按需推翻)	21
7. 待深化 / 待试的开放点 (v0.5 更新)	21
DD-01 详细设计: 领域对象与事件账本	23
0. 设计输入与要回答的问题	23
1. 第一性: canonical vs derived	23
2. 领域对象模型	24
3. 事件账本	28
4. 关键场景走查 (trace 示例)	33
5. 派生投影清单 (derived, 全可重建)	34
6. 落地顺序指南 (不排期)	34
7. 开放决策 / TODO (进实现前要钉死)	34

8. 面试叙事（一句话）	35
DD-02 详细设计：阶段状态机与 Harness（编排器）	37
0. 设计输入与要回答的问题	37
1. 两个正交维度：phase × run_status	37
2. 阶段属性表（每阶段一份配置，数据驱动）	38
3. 合法迁移表	39
4. 门控模型：机器门槛 × 人工门 × 研究闸门	40
5. 模型 vs 编排器：ActionProposal 契约	40
6. 主循环：research_turn（含两层循环）	41
7. 外层：事件驱动的会话与中断（steering / follow-up / 打断）	42
8. 健康探针放置（触发点）	42
9. 并行与顺序规则（M0-M3 内仍是单 agent）	43
10. M0 最小可跑 loop（指南，不排期）	43
11. 开放决策 / TODO	43
12. 面试叙事（一句话）	43
DD-03 详细设计：上下文 / 投影 / 记忆	45
0. 设计输入与要回答的问题	45
1. 一条主线：canonical / 投影 / prompt 三者不互相污染	45
2. 上下文分层（ContextPack 的内容结构）	46
3. ContextPack 结构 + build_context 契约	46
4. token 预算模型（预算 = 每步决策输入，不是报错才看）	47
5. 大工具结果：摘要进上下文、完整进账本、细节按需读	48
6. 压缩 = 语义 checkpoint（不是删消息）	48
7. 图片 / 视觉进上下文（按模型能力裁剪）	48
8. 记忆四层落地（“项目记忆/偏好”与“证据库”分开）	49
9. 与 DD-01/DD-02 的接线	49
10. M0 最小实现指南	49
11. 开放决策 / TODO	50
12. 面试叙事（一句话）	50
DD-04 详细设计：工具契约与权限（Policy）	51
0. 设计输入与要回答的问题	51
1. 工具 = 受控能力对象（不是函数）	51
2. 工具分层（Registry / Router / Runtime）	52
3. 工具清单与策略映射（含 stata-mcp 10 工具，已实锤）	52
4. Policy 裁决：三拆 + 固定顺序	54
5. 权限维度矩阵（组合而非开关）	54
6. 高风险入口（作者 do-file / 联网复现 / 长命令）	55
7. 接线与事件	55

8. M0 最小实现指南	55
9. 开放决策 / TODO	55
10. 面试叙事（一句话）	55
DD-05 详细设计: Writer + Validator（证据→实证初稿）	57
0. 设计输入与要回答的问题	57
1. 证据→初稿的管线（写入权分离落地）	57
2. 表格 = esttab 语义对象（渲染与反查共用同一模型）	57
3. 内容产出	58
4. Validator 规则集（写稿前/稿中必跑）	58
5. Writer 流程（分节，学顶刊）	59
6. Word 产出链路（选型仍开放，给倾向）	60
7. 接线	60
8. 开放决策 / TODO	60
9. 面试叙事（一句话）	60
DD-06 详细设计: 评测（五层 L0-L4 + 复现 benchmark）	61
0. 设计输入与要回答的问题	61
1. 评测总体（跑什么、在哪跑）	61
2. L0 契约（每次提交）	61
3. L1 组件（gold set + graded match）	61
4. L2 轨迹（trace grader + 规则）	62
5. L3 结果（单元格级数值评分）→复现 benchmark	62
6. L4 研究质量（盲评专家 + 校准过的 LLM judge）	62
7. 对抗集（自动化 eval 用例，validator/writer/权限都要抓）	63
8. 落地清单 / TODO	63
9. 面试叙事（一句话）	63
DD-07 详细设计: 文献管线（双 RAG）与 skill 规范	65
0. 设计输入与要回答的问题	65
1. 双库与信任角色（回顾，本设计给落地）	65
2. 摄取版本链（可溯源、可重建、可重跑）	65
3. 解析管线（含中文/扫描）	66
4. 混合检索 + rerank（落地）	66
5. citable 纪律（引用溯源）	66
6. 联网补文献（§4.10 接线，可选）	66
7. skill 规范（文件化方法论 = 版本化政策包）	66
8. 目录/接口	67
9. 开放决策 / TODO	67
10. 面试叙事（一句话）	68
设计审计: 按 Stata 实证研究实际需求回看 SPEC v0.5 / DD-01 / DD-02	69

0. 一句话结论	69
1. 两个能力问题的直接答复	69
2. 审计发现（按域）	70
3. 修补动作清单（按严重度）	71
4. 开放问题（要决策）	72

实证研究 Agent ——项目交接总览 (v0.5)

本目录 = agent 项目的工作区。开新 **session** 研究时在这个目录打开（会自动读 CLAUDE.md），思路都在本 README + design/、research/ 里，不用依赖旧对话。关联：MCP 执行层已完成并发布——<https://github.com/horizonial/stata-mcp>（本地 C:\Users\user\stata-mcp）。agent 通过 MCP 协议消费它跑 Stata。版本基线：**v0.5 = v0.4 + 合并成熟 coding agent 借鉴项 (2026-09-07, 来源 research/borrow_from_coding_agents.md, B+C 全并)**；**v0.4 = v0.3 + codex 架构深研 (research/codex_report.md)**。详细变更见 design/SPEC.md 顶部”修订记录”；本文档已同步 v0.5。

0. 一句话定位

研究伙伴 / 方法顾问型 **agent**（不是全自动写论文机）：学者自己想 idea、找文献、找数据放进约定文件夹；agent 与学者对话式判断可行性、指出缺什么、反复用 Stata 试 spec，最终产出实证部分的论文初稿（中文、Word、数字真实可溯源、引用真实可溯源、文风模仿顶刊）。

v0.4 一句话（面试叙事）：把一次实证研究变成可恢复、可复核、可追责的证据生产流程——以研究问题为中心、以 Stata 结果为数值真相、以文献原文为引用真相、以事件台账为过程真相。架构 = 确定性骨架（阶段状态机/权限/证据/副作用做软件规则）+ 概率性推理（选题/解释/候选方案留给模型）。

1. 已钉死的决策（多轮访谈 + v0.4 修订，见 SPEC §1）

维度	决策	关键理由
角色	强交互研究伙伴，对话式判断可行性	学者主导 idea/资料，agent 是方法顾问 + 执行者
形态	单 agent 循环起步，预留多 agent	先证明单 agent 出东西；多 agent 为解”上下文打架”
编排	确定性阶段状态机 + 阶段内受约束单 agent (v0.4)	状态迁移只归编排器；多 agent 需评测证明才启用
终点	idea →数据→实证→稳健性/机制/异质性→实证部分初稿	引言可浅，方法 + 结果 + 解读完整

维度	决策	关键理由
大脑	LLM 聚合层, provider 可扩展, 先 deepseek	按能力画像 + 契约测试编程, 不按模型名; 换模型只改配置
交互/UI	强交互 + 聊天 UI (后置); 可打断可自动	人工确认 = 持久化 approval 事件, 不只聊文本本
trace	结构化全链路 (decision_summary/action/tool_result/结论), UI 可回放; 不存思维链 (v0.4)	信任 + 调试 + 续跑; 可解释性靠”为何选它” 不靠回放 CoT
状态	SQLite events 表 (append-only) = 唯一事实源 + 物化视图 (v0.4)	jsonl 仅导出; 唯一真相源、可回放续跑
方法论	skill 文件化 (DID/机制异质性/稳健性/出表)	版本化政策包, 不写死代码, 可插拔扩展
文献	双独立库: ①顶刊库→学风②项目库→变量选取/引用 (带 bibtex)	信任角色类型化 (v0.4): style_only / citable_evidence / background
数值接地	EvidenceBundle : 先证据包后渲染 (v0.4)	正文数字/引文只引用已验证 claim, 不允许现编
隐私	三档模式: local_strict / approved_remote / mixed (v0.4)	fallback 不得跨隐私边界 (本地 embedding ≠ 全链路本地隐私)
初稿	中文 + Word (rtf 表嵌 Word), LaTeX 次要	学者要在 Word 里继续编辑
数据	不限定单目录, agent 需通用文件读取 (受 MCP 路径审计约束)	学者指出数据位置
Stata 并发	走 MCP 协议连 stata-mcp 一次一个 idea 工作区	职责清晰、可换后端 专注推进

2. 面试/工业级四维 (已定设计, 见 SPEC §4.9)

- 上下文与记忆: 分层 (线程状态 / 项目记忆 / 证据库 / 遥测); 工具大结果进事件表/证据库、上下文只放摘要; 每 phase 写结论压缩上下文。金句: *agent* 不靠记住, 靠随时能查——*RAG-over-own-history*。
- 安全: 外部输入 (文献/数据/粘贴) 不可信、与指令隔离 (信任标签); Stata 全走 MCP 受限模式不裸执行; 隐私三档模式 (v0.4)。金句: 最危险的是文献里的注入被当成你的指令, 最隐蔽的是 *fallback* 偷偷变成数据出境。
- 异常/兜底: 每层失败有明确降级 + 幂等键 (operation/attempt/语义哈希) + 恢复协议 (稳定阶段续跑、先核未决副作用再决定) + validator 不硬出 + 预算超停。金句: 超时是”不知道成功没”, 不是”肯定没成功”。
- 评测 (杀手锏): 实证有 ground truth ——复现≥ 3 篇异质论文当 eval + 五层 **L0-L4** (契约/组件/轨迹/单元格数值/专家盲评), 遮蔽答案、防污染。金句: 通用 *agent* 苦于没标准答案; 实证 *agent* 有——复现论文就是 *eval*, 数字对不上就是失败。

深做的 **2-3** 亮点 (v0.4 收敛, 别夸大): ① 数字来源闭环——每个正文数字绑定 Stata result_id + command_hash +

data_signature + do_file, 写作器只渲染已验证 claim; ② 研究决策可查询——规格选择/人工确认/拒绝理由/反事实分支 = 一等对象, 能答”为什么是这个模型”; ③ 实证社会科学单元格级复现协议——五层评测的 L3。混合 RAG/事件溯源/OTel/Chroma 等是成熟构件, 做扎实即可不当卖点。

3. 技术选型 (v0.4, 见 SPEC §1.5)

- 协议层: PydanticAI (工具循环/结构化输出/schema 校验) ——定位收窄为类型化模型/工具协议层, 不垄断业务状态与恢复语义
- 编排: 自写确定性阶段状态机 (阶段迁移/权限/人工门/失败策略) + PydanticAI 跑阶段内 agent 循环 (LangGraph 现不引入, 触发条件见 SPEC §1.5)
- 状态: SQLite events 表 (append-only, 唯一事实源) + 物化视图 (当前状态) + 快照/upcaster ——可查询/回放/审计/续跑; jsonl 是导出玩具
- 契约: agent → LLM 关键产物全 Pydantic model, 不过重试; schema 保”形状正确”, EvidenceBundle/validator 保”事实正确”
- 模型适配 (v0.4 P1): ModelCapabilityProfile + provider 契约测试; mock LLM provider (录 replay), harness 确定性单测
- 数值/引用接地 (v0.4) : EvidenceBundle (NumericClaim/ CitationClaim/ ResearchDecision + render_policy), 写作只渲染已验证 claim
- 双 RAG (重点 + 保留 grep): 向量检索 + grep/BM25 精确并查 (混合) + 可选 rerank; 分块级引用溯源 (块带 doc_id+ 页/章节/span); 信任角色类型化 (style_only/citable/background); Embedder/VectorDB 接口抽象 (Chroma 起步, 换型看指标不看篇数)
- 可观测 (v0.4 P1): SQLite 业务账本 + OTel → 现成后端 (Langfuse/LangSmith/Logfire), 两本账稳定 ID 关联, 默认不采内容
- 一次一个 idea 工作区; UI 后置但事件表数据模型现在定 (UI 只读物化视图 + 发消息)

4. 架构分层 (见 SPEC §3)

UI(后置: 聊天/进度/trace 回放, 只读物化视图+发消息)

↓

确定性阶段状态机(自写: IDEA→...→VALIDATION; 阶段迁移/权限/人工门/失败策略 只归它)

↓

受约束的单 agent 循环 PydanticAI(阶段内 plan→act→observe→reflect + skill 加载 + 预算, 每步写事件)

↓

能力层(注册式): stata_mcp_client(策略映射) / file_reader / ledger(events+物化视图)

|

/ rag(双库, 信任角色) / evidence_builder / writer(只渲染已验证) / validator(claim-to-source)

↓

RAG 内部: Retriever(混合: 向量+grep/BM25+可选 rerank) → VectorDB(Chroma 起步, 可换) + Embedder(可换)

↓

事件账本 SQLite(events append-only+物化视图+快照) 遥测 OTel→现成后端

↓

LLM 聚合: ModelCapabilityProfile + provider 接口 (deepseek 起步+mock); 隐私三档门(local_strict/approved_remote/mixed)

5. 一个 idea 的工作流（见 SPEC §2，跑在阶段状态机上）

阶段机：IDEA → LITERATURE → DESIGN → DATA → ESTIMATION → ROBUSTNESS → WRITING → VALIDATION → DONE；
每阶段有 `allowed_tools/human_gate`/失败策略，只有编排器能提交迁移。

学者想 idea → 找文献放库 → 找数据 → 起工作区告诉 agent → [0 建档: idea 声明 + 建 events 账本 + 载 skill] → [1 对话式可行性: 反问 → 查项目库 (citable) → 查数据 → 给”可行/缺数据/缺文献”，approval 事件持久化] → [2 实证设计与跑: 按 skill 提 spec → MCP 跑 → 结构化结果 + provenance 进证据库 → numeric_claims → 讨论迭代] → [3 稳健性/机制/异质性] → [4 实证初稿: 学顶刊文风 (style_only) → 引项目库真实文献 (citable) → EvidenceBundle 渲染 → validator 过数字/引用/复现 → 中文 Word] → 学者改、迭代 → 存档 (events 导出 + do-files + 初稿 + 引用清单)。

6. 路线（给 codex/新 session 的切法，v0.4 合并 P1/P2，见 SPEC §5）

- **M0** 骨架：阶段状态机（最小）+ PydanticAI 协议层 + LLM 聚合 (deepseek+mock) + **SQLite events** 表 (schema/版本/幂等骨架) + 物化视图 + 工作区脚手架 + CLI 对话（一句一问、都进表、不存思维链）
- **M1** 可行性：file_reader + 双 RAG 落库（摄取版本链/稳定 chunk_id/source_role）+ 混合检索 + 50-100 条 gold set 消融 + 对话式可行性 + 隐私三档默认 **local_strict** + OTel 埋点；验收 = 真实 idea 跑通
- **M2** 实证：连 stata-mcp + 策略映射/错误分类 + skills(先 panel_id) + approval 门控事件 + 恢复协议（幂等 + 未决副作用核对）；验收 = 复现 1-2 篇基准主表
- **M3** 初稿：EvidenceBundle + writer + validator + Word/rtf + 顶刊文风 (style_only 隔离)；验收 = 对抗注入 validator 全抓 + 复现集扩 ≥ 3 篇异质，跑 L0-L3
- **M4** 观测/UI：聊天 + 进度 + trace 回放 + OTel 接现成后端
- **P2**（评测证明需要再做）：多 Agent 文献广搜 / Qdrant / Temporal-DBOS / 自研 Dashboard

7. 待深化/待试（见 SPEC §7 与 research 提示词）

v0.4 已关闭：ledger.jsonl vs SQLite 冲突、thought 持久化、本地隐私 vs DeepSeek 矛盾、双库只靠 collection 隔离、validator 事后校验。待深化集中在：阶段状态机细节、provider 契约测试、事件 schema 演进 (upcaster/快照)、恢复协议落地、中文 PDF 扫描版 OCR、Word 产出链路、检索 gold set 与 hybrid 消融、评测集 ≥ 3 篇异质论文 + 专家容差。深研任务书在 research/PROMPT_codex_research.md；**codex** 交付报告在 research/codex_report.md（已完成一轮，**P0** 已并入 **SPEC**）。

8. 文件导航

- CLAUDE.md ——新 session 自动读的速览
- README.md ——本文档，交接总览 (v0.5)
- design/SPEC.md ——完整设计规格 **v0.5**（决策/架构/四维/路线/开放点；顶部有 v0.4/v0.5 修订记录）
- design/dd-01-domain-events.md ——详细设计 01：领域对象 (ResearchState/ResearchSpec/ExperimentFamily/Run/Claim/EvidenceCard) 与事件账本（事件目录/schema/reconcile/分支/fence/恢复/健康探针）
- design/dd-02-phase-machine-harness.md ——详细设计 02：阶段状态机 + harness (phase×run_status、阶段属性表、迁移表、门控模型、ActionProposal 契约、research_turn 循环、预算/steering/健康探针、M0 loop 指

南)

- design/dd-03-context-memory.md —详细设计 03: 上下文/投影/记忆 (ContextPack 分层、canonical →投影→prompt 不互相污染、token 预算、压缩 = 语义 checkpoint、大结果摘要 + 按需读、项目记忆 vs 证据库、图片/vision 裁剪)
- design/dd-04-tool-permission.md —详细设计 04: 工具契约与权限 (ToolContract 四段、stata-mcp 10 工具真实策略映射、Policy 固定裁决顺序、权限维度矩阵、author do-file/联网高风险入口、M0 指南)
- design/dd-05-writer-validator.md —详细设计 05: Writer + Validator (证据→初稿管线、esttab 表语义解析器、表/图/Table1 渲染、Validator 规则集、分级证据门槛 L-C/L-R/L-D、Word 链路)
- design/dd-06-eval.md —详细设计 06: 评测 (五层 L0-L4 实现、复现 benchmark ≥ 3 篇异质、遮蔽答案/防污染、mock 确定性替身、对抗集、LLM judge 校准)
- design/dd-07-rag-skills.md —详细设计 07: 文献管线 + skill 规范 (双库摄取版本链/抽取 QA/OCR 接线、混合检索 +rerank、citable 纪律、skill 文件 schema 与激活 preflight)
- design/audit-stata-practice.md —Stata 实证需求审计 (2026-09-07): prep/pipeline、env_sig、表语义 locator、图证据、门控探索环、联网/图片能力决策 (S1 已修入 DD-01, S2 修入 DD-02/SPEC)
- research/PROMPT_codex_research.md —交给 codex 的深度调研任务书
- research/codex_report.md —codex 调研交付报告 (2026-09-03, SPEC v0.4 的修订依据, 从 docx 转存)
- research/borrow_from_coding_agents.md —借鉴引入笔记 (2026-09-07, 从 pi/codex/claude-code/claw-code 学习册提取可借鉴项→SPEC 映射 + codex 记忆系统复查; v0.5 候选清单见其 §6)
- MCP 执行层: C:\Users\user\stata-mcp (DESIGN.md 记录 7 轮审计; README 中英优劣势)

实证研究 Agent ——设计规格 (v0.5, 工业级)

用途: 交给 codex 深化实现。本文是需求与架构的”钉子”, codex 在此基础上细化技术选型、产出代码。有分歧时以本文为准, 存疑点标注 [TODO]。关联项目: C:\Users\user\stata-mcp (Stata MCP 执行层, 已完成)。本 agent 通过 MCP 协议驱动它跑 Stata。版本基线: **v0.5 = v0.4 + 合并成熟 coding agent (pi/codex/claude-code/claw-code)**借鉴项(2026-09-07, 来源 ../research/borrow_from_coding_agents.md §3/§6)。评审对象路径以本文件 design/SPEC.md 为准。

修订记录

v0.4 采纳项 (codex 架构深研, 2026-09-03, 来源 ../research/codex_report.md)

- **P0-1** 状态唯一真相: SQLite **events** 表 (**append-only**) 是唯一事实源; 当前状态由物化视图 (可重建) 给出; ledger.jsonl 降级为导出, 不参与恢复。
- **P0-2** 编排: 单 agent 循环放进确定性阶段状态机内的受约束循环; 状态迁移只归编排器。多 agent M0-M3 不启用, 启用需评测证明 + ADR。
- **P0-3** 推理产物: 不存原始 thought (思维链), 存可审计 **decision_summary** + 证据 ID + 候选/选中/拒绝理由。
- **P0-4** 数值/引用接地升级: 写作 = **EvidenceBundle** (先证据包后渲染); 正文只许引用已验证 claim ID。
- **P0-5** 事件与可靠性: 事件 schema/版本、幂等键 (operation/attempt/语义哈希)、未决副作用与恢复协议; upcaster + 快照。
- **P0-6** 隐私三档: local_strict / approved_remote / mixed_sanitized; fallback 不得跨隐私边界。
- **P0-7** 双 **RAG** 信任角色: style_only / citable_evidence / background_only 类型化。
- **P0-8** 评测升格: 五层 L0-L4, 复现 ≥ 3 篇异质论文、遮蔽答案、专家容差。
- P1 (M0-M1 完成项) 与 P2 (评测门控项) 已并入 §5/§7。

v0.5 采纳项 (借鉴 coding agent, 2026-09-07, 来源 ../research/borrow_from_coding_agents.md)

- **N1 events** 补齐可重建字段 + **reconcile** (B1, claw LaneEvent): 序号、来源、置信度、fingerprint、parent_id、attempt 状态; “终态后的不确定事件”要 reconcile, 否则多来源会重复/矛盾, events 当不成唯一真相。
- **N2** 压缩/恢复后健康探针 (B2, claw): 恢复 ≠ 读文件回来, 先跑最小一致性检查再进 Loop。

- **N3** 工具闭合不变量 + 半截不执行 + 取消为正常态 (B3, claude-code/pi): 每个 tool_use 必有带 ID 的结果; 取消/权限拒绝 = cancelled 正常结果回模型 (blocked 态); stopReason=length 的半截参数不得执行。
- **N4 token 预算 = step 决策输入** (B4, codex/claude-code): auto-compact 缓冲阈值, 提前触发而非超限补救。
- **N5 研究完整性闸门 + 研究者自由度账本** (B5, claude-code/架构论文): 覆盖样本/换 FE/换聚类/删异常值单独审批记录, 独立于隐私三档。
- **N6 结果三层契约** (B6, codex): 机器可验证层 / 模型可读层 / 人类附件层, 固定进 EvidenceBundle。
- **N7 分级证据门槛 + 显式 Validation 状态** (B7, claw/codex): 可行性 vs 初稿是不同证据等级; “模型说可交付”不算完成。
- **N8 上下文 = 多级投影** (B8, claude-code/pi): manifest → 候选 → 按需读; 投影不覆盖 canonical。
- **N9 ResearchState 内容层研究状态** (C1, claude-code/架构论文): 样本口径/变量角色/识别假设/证据引用结构化, 独立于阶段机。
- **N10 Claim/EvidenceCard locator + ExperimentFamily** (C2, 架构论文/codex): 防只记录喜欢的规格; 追溯粒度到页码/表号/行列。
- **N11 写入权分离** (C3, codex/claude-code): EvidenceBundle/EvidenceCard 只能由 Validator + Stata 执行事实生成, 模型不得直接写 claim。
- **N12 探索性 vs 确认性分离 + confirmatory lock** (C4, 架构论文): 看过结果后再改动必须 amendment 记录。
- **N13 分支语义 + rollback 分离上下文与文件副作用** (C5, pi/codex): 换 spec 即分支; “隐藏失败尝试” ≠ “删除写出的文件”。
- **N14 writer lease + revision/fence** (C6, pi): 防崩溃恢复后旧进程覆盖新状态。
- **N15 双投影事件溯源** (C7, codex, 演进方向): events 存核心事实, 运行时投影为模型 history / 用户 turn / 评测 event 三视图。
- **N16 阶段 0: 无 Agent 可信实验内核** (C8, 架构论文, M0 可选前置)。
- **N17 长期记忆借鉴 codex 记忆管线** (截断式, 见 §4.9.1)。

0. 一句话定位

一个研究伙伴 / 方法顾问型 agent (不是全自动写论文机): 学者自己想 idea、找文献、找数据, 放进约定文件夹; agent 与学者对话式判断 idea 可行性、指出还缺什么数据/文献、反复用 Stata 试不同 spec, 最终产出实证部分的论文初稿 (中文、Word、数字真实可溯源、引用真实可溯源、文风模仿顶刊)。

v0.5 深化定位 (面试叙事): 把一次实证研究变成可恢复、可复核、可追责的证据生产流程——以研究问题为中心、以 Stata 结果为数值真相、以文献原文为引用真相、以事件台账为过程真相。架构 = 确定性骨架 (阶段状态机/权限/证据/副作用做软件规则) + 概率性推理 (选题/解释/候选方案留给模型)。**v0.5** 补一条硬规则: 模型的想法不是统计事实——模型只能提议, 进入正文的数字/引文只能由”验证器 + Stata 执行 + 真实文献块”生成。

1. 已钉死的决策 (多轮访谈 + v0.4/v0.5 修订)

维度	决策	说明
Agent 形态	单 agent 循环起步，预留多 agent	先证明单 agent 能出东西；多 agent 是为解决”研究执行/写作上下文打架”，不是目标
编排	确定性阶段状态机 + 阶段内受约束单 agent 循环	状态迁移只归编排器；多 agent M0-M3 不启用，仅当独立并行检索/权限隔离/上下文隔离经评测证明有收益才启用（ADR）
workflow 终点	idea →数据→实证设计→跑主回归→稳健性/机制/异质性→ 实证部分初稿	引言/综述可浅，实证方法 + 结果 + 解读要完整
大脑	LLM 聚合层，多 provider 可扩展，先 deepseek	按能力画像 + 契约测试编程，不按模型名编程
交互	强交互 + 聊天 UI （后置）；可打断、可自动	人工确认是持久化的 approval 事件，不只是聊天提示
可行性	对话式：反问澄清→查项目文献库→查数据→给”可行/缺数据/缺文献” + 下一步	学者持续迭代思考
trace/审计	结构化全链路（decision_summary/action/tool_result/结论），可回放；不存原始 thought	可解释性靠”为何选它”，不靠回放 CoT
状态	events 表（ SQLite append-only ）= 唯一事实源；物化视图 = 当前状态	v0.5: 加可重建字段/序号/parent/reconcile (N1)；分支语义 (N13)
研究状态	ResearchState 内容层：样本口径/变量角色/识别假设/证据引用独立结构化（v0.5 N9）	阶段机管”走到哪”，ResearchState 管”当前用什么样本/模型/结论前提”
推理产物	不存思维链；存 decision_summary + 证据 ID + 候选/选中/拒绝	模型想法≠统计事实，写入权分离（v0.5 N11）
记忆	分层记忆：线程状态/项目记忆/证据库/遥测	证据库 = 可版本化长期，走 EvidenceBundle/溯源；偏好记忆借鉴 codex 两阶段（N17）
方法论	skill 系统：内置基础（面板/DID/机制异质性/稳健性/出表）+ 可扩展	skill = 版本化政策包（meta+ 触发 + 前置检查 + 步骤 + 规则 + 禁用 + 示例）
文献	两独立库：①顶刊库→学文风②项目库→变量/引用（带 bibtex）	信任角色类型化：style_only / citable_evidence / background_only
初稿	中文 + Word 为主（rtf 表嵌 Word），LaTeX 次要	文风模仿顶刊库

维度	决策	说明
数据	不限定单目录；agent 需通用文件读取 (受路径审计约束)	学者指出数据位置
Stata	走 MCP 协议连 stata-mcp	stdio 先, HTTP 后
隐私	三档运行模式: local_strict / approved_remote / mixed_sanitized	fallback 不得跨隐私边界
研究完整性	研究闸门: 改样本/识别策略/FE/聚 类/删异常值单独审批记录 (v0.5 N5)	与隐私三档 (数据安全) 分离
并发	一次一个 idea 工作区	专注推进, done 存档再开

1.1 v0.5 借鉴采纳速查 (来源与落点)

采纳	来源 (借阅笔记 §3/§6)	落实在本文
events 可重建字段 + reconcile	B1	§4.4
恢复/压缩后健康探针	B2	§4.9.3
工具闭合不变量 / 半截不执行 / 取消正常态	B3	§4.4、§4.9.3
token 预算 = step 决策输入	B4	§4.9.1
研究完整性闸门 + 自由度账本	B5	§4.9.2
结果三层契约	B6	§4.8
分级证据门槛 + 显式 Validation	B7	§4.3、§4.8
上下文多级投影	B8	§4.9.1
ResearchState 内容层	C1	§4.3.1
Claim/EvidenceCard + ExperimentFamily	C2	§4.8
写入权分离	C3	§4.8
探索/确认分离 + lock	C4	§4.9.5
分支语义 + rollback 边界	C5	§4.4、§2
writer lease + fence	C6	§4.4
双投影事件溯源 (演进)	C7	§4.4
阶段 0 无 Agent 内核	C8	§5

1.5 技术选型与面试叙事 (工业级)

每个选型能讲清 trade-off; 不堆功能, 深做 2-3 个差异化点。成熟构件 (混合 RAG、事件溯源、Pydantic、OTel、Chroma/Qdrant) 做扎实即可不当卖点。

层	选型	理由 / 面试叙事	状态
协议层（工具循环/结构化/流式）	PydanticAI	类型化模型/工具协议层，不垄断业务状态与恢复语义	✓ 已定
编排	自写确定性阶段状态机 + PydanticAI 跑阶段内 agent	状态迁移协议独立成产品资产；未来跨天/分布式再换 LangGraph/DBOS/Temporal	✓ 已定
状态/记忆	SQLite events 表 (append-only) + 物化视图 + 快照 + 可重建字段/reconcile (N1)	events= 唯一事实源；单 writer 假设显式记录	✓ 已定
研究状态	ResearchState 内容层对象 (N9, v0.5)	压缩/续跑时保留”结论前提”	• 待细化 schema
契约	agent → LLM 关键产物全 Pydantic model , 校验不过自动重试	schema 保”形状正确”, EvidenceBundle/Validator 保”事实正确”	✓ 已定
证据链	Claim / EvidenceCard locator / ExperimentFamily (N10, v0.5)	追溯粒度到页码/表号/行列；防只记录喜欢规格	• 待细化
模型适配	ModelCapabilityProfile + provider 契约测试	按能力编程不按模型名；换模型跑契约回归	• P1
可测性	mock LLM provider (录 replay), harness 确定性单测	能回答”怎么保证 agent 逻辑对”	✓ 已定
RAG	混合检索（向量 + grep/BM25 + 可选 rerank）+ 信任角色	语义召回找意思、词法守住名字；rerank 由 gold set 定	✓ 已定
Embedding	Embedder 接口抽象，可换 (bge-m3/大陆 API)	用本项目数据与更新候选比，不写死	✓ 接口抽象
向量库	VectorDB 接口抽象； Chroma 起步	换型看延迟/召回/并发/运维，不是篇数 (Qdrant P2)	• 候选
文献解析	中英 PDF →分块，记页/章节/bbox	分块级可溯源	• 待试
引用粒度	分块级（段/页 + span），非文章级	不能把 A 文结论安到 B 文	✓ 已定

层	选型	理由 / 面试叙事	状态
数值/引用接地	EvidenceBundle : 先证据包后渲染 + 写入权分离 (N11)	模型只能提议, claim 只能由 Validator+Stata 事实生成	✓ 已定
完成定义	分级证据门槛 (可行性/主回归/初稿分级, N7)	模型说完成 ≠ 完成, 查 do 文件/rc/结果一致性	✓ 已定
规模余量	按几百篇预留	索引/去重/多路召回按几百设计	✓ 已定
Stata 执行	stata-mcp (走 MCP)	上层做 10 工具策略映射 + 错误分类 (P1)	✓
可观测	SQLite 业务账本 + OTel → 现成后端	两本账稳定 ID 关联, 默认不采内容	• P1
UI	后置, 但事件表数据模型现在定	UI 只读物化视图 + 发消息	✓ 已定

深做的 **2-3** 个差异化亮点: ①数字来源闭环 (数字→ result_id + command_hash + data_signature + do_file → EvidenceCard locator) ②研究决策可查询 (ResearchState/自由度账本/反事实分支 = 一等对象) ③实证社会科学单元格级复现协议 (**L3**)。其余做扎实即可。

2. 工作流 (一个 idea 的生命周期, 跑在阶段状态机上)

阶段状态机: IDEA → LITERATURE → DESIGN → DATA → ESTIMATION → ROBUSTNESS → WRITING → VALIDATION → DONE。每阶段声明 entry/exit conditions、allowed_tools、exit_schema、human_gate、max_attempts、failure_policy; agent 只能在阶段内循环, 只有编排器能提交状态迁移; 中途失败进失败态 (可回退/重做)。阶段内额外维护 **ResearchState** (当前样本/变量/识别假设/证据引用, 见 §4.3.1) ——阶段机管”走到哪”, ResearchState 管”现在用什么”, 两者都随 events 重建。

- [学者] 想 idea → 自己找文献放库 → 自己找数据 → 起一个 idea 工作区
↓ 告诉 agent: idea + 数据位置提示
- [0. 建档(IDEA→DESIGN)] 初始化工作区: idea 声明 + events 账本/物化视图 + ResearchState + 载 skill
↓
- [1. 对话式可行性判断(DESIGN 内循环)] ← 证据等级: 可行性结论
agent 反问澄清 (研究问题/因果链/可观测变量...)
→ 查项目文献库 (只从 citable_evidence 取证) → 查数据支撑
→ 结论: "可行 / 缺数据(缺什么) / 缺文献(建议找哪类)" + 下一步 (approval 事件持久化)
↓ (学者补料或确认)
- [2. 实证设计与跑(ESTIMATION)] ← 证据等级: 单次运行可验证
agent 按 skill 提 spec (候选 proposal) → MCP 跑 Stata (stata_run/load/inspect...)
→ 结构化结果 + provenance 进证据库 → Validator 生成 EvidenceCard → ResearchState/ExperimentFamily 更新
→ 换 spec 即分支 (共享不可变过去 + suffix), 失败尝试不覆盖, 只记录
↓
- [3. 稳健性/机制/异质性(ROBUSTNESS)] (按加载的 skill 走一遍; 首个可并行点, 仍单 agent)
↓
- [4. 实证初稿(WRITING→VALIDATION)] ← 证据等级: 可交付初稿
agent 读顶刊库学风(style_only) → 引项目库真实文献(citable_evidence)

→ 从 EvidenceBundle 渲染数字/表格（不现编；写入权分离）→ validator 过数字/引用/复现 → 中文 Word
↓ (学者改, agent 按反馈迭代; 改稿再走 VALIDATION; 看过结果后的改动走 amendment 记录)
[完成] 工作区存档 (events 导出 + 快照 + do-files + 初稿 + 引用清单)

3. 系统架构

UI (后置 M1+): 聊天 / 进度 / trace 回放 / 结果预览	
——只读物化视图 + 发消息, 与 harness 解耦	
确定性阶段状态机 (自写): IDEA→...→VALIDATION; 每阶段	
entry/exit • allowed_tools • human_gate • failure_policy;	
状态迁移/权限/审批只归它; 含 blocked 态与 Validation 状态	
受约束的单 Agent 循环 (PydanticAI, 阶段内):	
plan → act → observe → reflect; 模型只产出候选 proposal;	
每步写事件; token 预算/健康探针为决策输入	
能力层 (注册式)	
• research_state → ResearchState 内容层读写 (§ 4.3.1)	
• stata_mcp_client → MCP 驱动 stata-mcp (策略映射)	
• file_reader → 读本地数据 (路径审计)	
• ledger → events + 物化视图 + 快照 + reconcile	
• rag (双库) → 顶刊[style_only • 文风]/项目[citable]	
• evidence_builder → 结果→EvidenceCard→Claim (只此能写)	
• writer → 只渲染已验证 claim → 初稿(Word)+rtf	
• validator → claim-to-source + 分级证据门槛	
RAG 内部: Retriever(混合: 向量+grep/BM25+可选 rerank)	
→ VectorDB(Chroma 起步,接口可换) + Embedder(接口)	
事件账本 SQLite: events(append-only,含 parent/fingerprint)	
+ 物化视图 + 快照 + 单写者租约(fence); 双投影为演进方向	
遥测 OTel → 现成后端 (Langfuse/LangSmith/Logfire, 可换)	
LLM 聚合层: ModelCapabilityProfile + provider 接口;	
deepseek 起步, 预留 qwen/kimi/claude; + mock(测试)	
└ 隐私三档门: local_strict / approved_remote / mixed	

4. 关键模块与数据契约

4.1 idea 工作区 (目录契约)

每个 idea 一个目录; 事件/状态在共享账本 (按 idea_id 分账), 目录不放 jsonl:

ideas/<idea_slug>/
├── idea.md # idea 声明: 问题、预期贡献、初始假设 (含 phase/ResearchState 版本)
├── data_refs.md # 学者告诉 agent 的数据位置/文件清单
├── dofiles/ # 复现用 do 文件 (含 MCP 提供的 provenance do_file)
├── outputs/ # 初稿、表格(rtf)、图; events 导出(ledger.jsonl 仅此用途)
└── notes/ # agent 与学者的讨论要点/待办

一次只开一个活跃工作区。

4.2 LLM 聚合层 + 能力画像 (llm.py / capabilities.py)

- 统一接口: chat(messages), chat_tools(messages, tools), chat_structured(messages, schema)
- provider 抽象: class LLMProvider(Protocol); DeepSeekProvider (OpenAI 兼容)
- **ModelCapabilityProfile**: provider/model/version; supports_tools / strict_schema / thinking_with_tools / streaming; max_context / max_output / json_schema_subset; known_quirks / tested_at / contract_test_version
- **Provider** 契约测试: 空参/非法枚举/并行工具/超长 schema/流式中断/工具结果回填/重试副作用; 锁定实际模型 ID 与 request ID; 模型升级必触发回归
- 隐私门: provider fallback 受 §4.9.2 三档约束——本地→远端是新的授权事件

4.3 Harness 与阶段状态机

- 阶段状态机 (确定性部分): IDEA → LITERATURE → DESIGN → DATA → ESTIMATION → ROBUSTNESS → WRITING → VALIDATION → DONE; 每阶段注册 entry_conditions / allowed_tools / exit_schema / human_gate / max_attempts / failure_policy。agent 提出候选迁移, 编排器提交。
- 状态集含正常失败分支 (v0.5 N3/N7): blocked (等外部输入/审批)、validating (显式 Validation 状态, 验证通过才发 EvidenceCard) 是正常态, 不是异常; 阶段迁移拒绝非法序列 (如未 started 就 completed、completed 后又 append attempt)。
- **skill** 加载: 按当前阶段 + 任务激活, 注入 system prompt。
- 人工门控: 可行性结论 / 主回归结果 / 稳健性方案 / 初稿前停等学者; 确认持久化为 approval 事件。
- **token** 预算: 作为每步决策输入 (N4), 含 auto-compact 缓冲阈值, 提前触发压缩而非超限补救。
- **checkpoint/续跑**: 从 events 重放最近稳定阶段 (含未决副作用核对 + 健康探针, §4.9.3), 不简单续写对话。

4.3.1 ResearchState (内容层研究状态, v0.5 N9)

独立于”阶段机”(控制流)的内容层对象, 回答”当前研究到底在用什么/信什么”:

```
ResearchState:
sample_definition    # 样本口径: 面板?样本区间?筛选规则(及各自版本/批准记录)
variable_roles       # 因变量/处理/协变量/聚类层级的变量映射(带构造 do-file 引用)
identification_assumptions # 识别假设(平行趋势?外生性?)与其依据文献/证据
spec_versions        # 当前主 spec + 已探索变体(ExperimentFamily 指针)
evidence_refs        # 当前结论依赖的证据 ID 集(EvidenceCard)
phase                # 与控制流同步但独立存储
```

- 由 events 重建 (物化视图的一种), 不依赖对话文本。
- 压缩/续跑/分支时保留的是它 + 证据索引, 不是对话摘要 (codex/claw 的 checkpoint 教训)。
- 每次 reflect 前把 ResearchState 摘要进 prompt (多级投影的”研究状态层”)。

4.4 事件账本 Ledger + 物化视图 (核心)

SQLite **events** 表 = 唯一事实源 (append-only); 当前状态 = 物化视图 (可重建); ledger.jsonl 只做导出。

events 表关键列 (v0.5 补齐 N1/N13/N14):

```
event_id(uuid) | idea_id | phase | event_type | schema_version | actor(user|agent|system)
| correlation_id | causation_id | operation_id | attempt_id | seq(单调序号)
| source(来源: agent/stata-mcp/user/system/validator) | confidence(置信度)
| fingerprint(去重指纹) | parent_id(分支/因果) | leaf_id(当前分支尾)
| payload(json) | created_at
```

事件种类 (kind=event_type):

```
{"ts":..., "kind": "agent_step", "decision_summary": "用 fe+cluster(id) 做主回归是因为...",
 "evidence_ids": ["res_..."], "candidates": ["fe", "fe+cluster"], "chosen": "fe+cluster", "rejected": {...}}
{"ts":..., "kind": "tool_call", "tool": "stata_run", "operation_id": "op_...", "attempt_id": 1,
 "semantic_input_hash": "sha256(...)", "side_effect_state": "committed", "args_hash": "...", "rc": 0,
 "structured_summary": "...", "cost": ...} // 闭合: 必有对应 tool_result (N3)
{"ts":..., "kind": "result", "result_id": "res_...", "hypothesis": "...", "conclusion": "...}
{"ts":..., "kind": "user", "text": "..."}
{"ts":..., "kind": "approval", "what": "采用 fe+cluster 做主回归", "status": "approved", "by": "user"}
{"ts":..., "kind": "provenance", "result_id": "...", "do_file": "...", "command_hash": "...,
 "data_signature": "...", "num_ref": "表 3-列 2", "spec_version": ...}
```

- 不存原始 **thought**; 推理只沉淀为 **decision_summary** + 证据 ID + 候选/选中/拒绝理由。
- 可重建字段 + **reconcile (N1)**: seq/source/confidence/fingerprint/parent_id 使多来源 (Stata 重试、人工批准、报告生成) 可去重、可按序重建; “终态后的不确定事件” 由 **reconcile** 逻辑判断 (查现状/补偿/人工), 不能盲信写入顺序。
- 分支语义 (**N13**, **N5** 相关): 一次 idea 内换 spec = 在 parent_id 树上新开分支 (共享不可变过去 + 自己的 suffix/leaf); 失败/被否的尝试保留, 只标记不覆盖。 **rollback** 只从“实验历史”隐藏, 不撤销文件系统——写出的 do-file/结果文件是否删除由显式策略决定, 二者不绑定。
- 物化视图 (随事件在单写者事务内原子更新投影): idea_state / current_phase / research_state (§ 4.3.1) / experiment_family / spec_version / result_index / citation_index / approval_log。
- 写并发 (**N14**): 单写者事务 + **writer lease** + 单调 **revision (fence)**, 防崩溃恢复后旧进程覆盖新状态 (长跑 Stata 场景)。
- 快照 + **upcaster**: 恢复从最近快照重放; schema_version 变化走 upcaster + 迁移测试。
- 双投影 (**N15**, 演进方向): events 存核心事实; 运行时投影为①模型 history (多级投影 § 4.9.1) ②用户可见 turn ③评测所需 event 三视图——先以物化视图落地, 标注为演进项不立即实现。
- 每个进初稿数字都要有 num_ref → result (数字接地, 见 § 4.8)。
- 遥测走 OTel (§ 6), 两本账分离。

4.5 双 RAG (混合检索, 分块级溯源, 信任角色类型化)

- 三信任角色: style_only (顶刊, 学风, 不得产生引用) / citable_evidence (项目/权威, 可 claim-to-span 引用) / background_only (理解用, 终稿不可直接引用)。
- 入库版本链: file_hash → parser_version → page/section/bbox → chunk_id(稳定) → source_role → embedding_version → index_version; PDF 更新后旧 chunk 失效保留历史。
- 检索 = 向量召回 + grep/BM25 精确并查 + (可选) rerank; 每块带 doc_id + source_role + 页/章节/span。

- M1 优先级: ①抽取质量/错页 QA ②元数据 + 稳定 chunk ID ③ 50-100 条 gold set ④ dense/BM25/hybrid/rerank 消融⑤换库/换 embedding 最后。
- 接口: Retriever.search(query, top_k, roles) -> list[Chunk]; VectorDB; Embedder。

4.6 Skills (文件化方法论 = 版本化政策包)

panel_did / mechanism_heterogeneity / robustness / regression_table_word / _template。每个 = meta + 触发条件 + 前置检查 + 步骤 + 规则 + 禁用条件 + 示例 + 版本; skill 声明 requires: {ados:[reghdfe, esttab, coefplot, ...], stata_min:"..."}, 激活时 preflight 查 which <ado>, 缺则提示安装/换机、不静默跑错 (否则污染"代码 vs 环境"归因, 审计 D2); skill 变更走版本, 供 L1/L2 评测与回归。

4.7 Writer (初稿) ——只渲染已验证证据

- 输入: 被"选定用于初稿"的 **Claim/EvidenceCard** 集 + 项目库引用 + 顶刊文风样本。
- 约束 (N11): 模型只能提议叙述; 正文数字只能引用已过 validator 的 NumericClaim (带 result_id), 引文只能引用 CitationClaim (带 chunk span); 模型不得现注入新数字/新引文。
- 输出: 中文实证初稿 (方法/结果/解读), 表格由数字渲染 (esttab → rtf / csv → Word), 每格带 num_ref。
- Word 生成: [TODO] 选型 (pandoc md → docx / python-docx / rtf 直出) ——目标"学者能在 Word 里继续编辑"。

4.8 EvidenceBundle + Claim/EvidenceCard + Validator (写稿前必跑)

证据链分层 (v0.5 N6/N10/N11) ——结果不是直接变文字, 中间经过两层:

```
Stata 结构化结果(带 provenance)    ← 机器层事实
→ Validator 生成 EvidenceCard      ← 精确 locator: 页码/表号/行列/result_id/data_signature
→ Claim (可核查命题)              ← 一句话研究级命题, Run→Claim→初稿段落
→ 初稿                             ← writer 只消费 Claim/EvidenceCard, 写入权分离
```

结果三层契约 (N6): 每个执行结果同时产出 - 机器可验证层: exit code / N / 系数 / SE / checksum (schema 只作用于这层, 只读) - 模型可读层: 摘要 / 警告 / 下一步 - 人类可追溯层: 完整 do-file / 日志 / 环境模型能解释机器层, 不能改机器层。

EvidenceBundle 中间对象:

```
class NumericClaim: # value, unit, display_precision, result_id, data_signature, command_hash, table_ref
class CitationClaim: # claim, chunk_id, doc_id, page/section, source_span, source_role, version
class EvidenceCard: # locator 到 页码/表号/行列/result_id; 由 Validator 签发 (N10/N11)
class ResearchDecision: # hypothesis, spec_version, approval_record
# render_policy: 正文只能引用已验证 claim_id; 写入权 = 仅 evidence_builder/validator, 模型只提议
```

ExperimentFamily (N10): 一次研究的所有运行按"尝试家族"分组——预计划 vs 探索、停止规则、多重检验记数、家族内全成员 Run、主结果选择理由。防只记录喜欢的规格; 主结果必须能回答"为何是这个 spec"。

分级证据门槛 (**GreenContract** 实证版, N7):

证据等级	何时用	门槛
L-C 可行性	DESIGN 出结论	项目库有据可查 + 数据变量可构造 + 学者确认
L-R 单次运行	ESTIMATION 出结果	rc=0 + 结构化结果 + 校验通过，可复现
L-D 可交付	WRITING → VALIDATION 出初稿	每表对应 dofile 可跑 + 数字/引用全溯源 + validator 过 + 对抗用例过

“模型说可交付”不成立——按 L-D 检查 do 文件/返回码/结果一致性才算完成 (N7)。

Validator 检：数字（命中 NumericClaim，原始值与展示值分离、渲染后反查）、引用（溯源到 citable_evidence 真实块）、复现（每表一个 dofile，签名一致）。schema 只保形状正确，validator 才保事实正确。

4.9 工程纵深四维（面试必答：上下文/安全/异常/评测）

4.9.1 上下文与记忆管理（不靠”记住”，靠”随时能查”）

- 分层记忆：

层次	内容	生命周期	是否可为证据
线程状态	当前阶段/待办/最近工具结果摘要	一次运行	否
项目记忆	已确认定义/偏好/变量映射/否决记录	idea 生命周期	仅作约束
证据库	文献原文块、Stata 结构化结果、数据签名、EvidenceCard	可版本化长期保存	是
遥测	token/延迟/错误/trace	运维保留期	否

- 多级上下文投影 (v0.5 N8)：完整历史/ResearchState 是规范 (events 可重建)；给模型的是投影——按层折叠（稳定规则→ ResearchState →证据摘要→原始材料按需读）。读文献/记忆用渐进披露：先 manifest 头→候选列表→按需读全文，“没选中也合法”；投影不覆盖 canonical。
- token 预算 (N4)：每步算 full active / auto-compact scope / model limit / buffer / remaining；提前触发压缩而非超限崩溃。
- 压缩 = 语义 checkpoint (N2/N8)：压成”研究问题/数据版本/已跑 spec 成败/已确认事实 (Claim)/识别风险/下一允许动作”，保留证据索引；写 events (boundary 事件)，不是删旧消息；压缩点在完整 turn/试算之后，绝不截断半条工具调用。压缩/恢复后跑健康探针再进 Loop (§4.9.3)。
- 长期记忆借鉴 codex 记忆管线 (N17，截断式)：可借鉴——两阶段解耦（逐 idea/phase 抽取 + 全局串行合并）、git-baseline diff 当”摄入 + 遗忘”信号、<citation> 引用协议让”用了哪条”可程序化、按产物类型用量遥测、consolidation 强沙箱、无变更最低信号门。不照搬——codex 记忆是偏好/流程记忆且会裁剪原始证据、抽取要联网；

我们 local_strict 下长期记忆只能存脱敏摘要/纯本地，证据库必须走 **EvidenceBundle**/溯源，绝不与偏好记忆混。

- 明令：不持久化原始思维链；短期上下文 ≠ 长期记忆。
- 金句：agent 不是记住一切，而是需要时从自己的长期记忆（事件表/证据库）检索——*RAG-over-own-history*，可复现、不爆窗口。

4.9.2 安全与风险（威胁模型 + 分层防线 + 隐私三档 + 研究闸门）

- 威胁模型：一切外部输入不可信（文献/数据/粘贴都可能 prompt 注入；OWASP LLM01/LLM06）。
- 信任标签：trusted_system / user_instruction / retrieved_untrusted / tool_result_verified；检索内容即使写着”运行 shell”也只是文本 span。
- 工具副作用分级（v0.5，与信任标签正交）：

工具/动作	默认策略
读授权文献、inspect data	自动允许
受限模式跑 Stata、读结果	阶段内允许，参数校验
覆盖原始数据、删除文件	禁止或单次强审批
联网下载	Deferred + 来源/版本校验
生成衍生变量/覆盖 do-file	workspace 审批
非受限 Stata、写授权目录外	人工确认或 allowlist
shell、凭据访问	禁止或单次强审批

- 研究完整性闸门（**N5**）——独立于隐私的”研究闸门”：改样本口径 / 换识别策略 / 换 FE / 换聚类 / 删异常值 / 主结果选择——这些默认”可执行但必须记录假设与变更（decision_summary + approval）“，且会让前后结果不可比的变更要显式提示。对应 研究者自由度账本（何时选、是否看过结果、依据）。
- 执行边界：Stata 全走 MCP restricted/路径审计；分隔符只是标签，真正边界 = 工具能力 + 文件权限 + 确定性策略（fail-closed）。
- 隐私三档（P0-6）：local_strict / approved_remote / mixed_sanitized；**fallback** 不得跨隐私边界；本地→远端是新授权事件（记 events）。
- 金句：最危险的是数据/文献里的注入被当成你的指令；最隐蔽的是 *fallback* 偷偷变成数据出境；研究 agent 最贵的是样本/识别策略被悄悄改掉——那要单独一道闸门。

4.9.3 异常处理与兜底（每层失败有明确降级 + 幂等 + 恢复协议）

层	失败	兜底
LLM	超时/限流/坏格式/拒答	分类：transient →重试退避 +jitter； validation/model_semantic →修 prompt 重出；policy/data →回 退/人工

层	失败	兜底
工具 (Stata)	崩溃/超时	MCP reset+ 重放; 超时≠失败——先查未决副作用
流程	中断/进程死	恢复协议: 读最后稳定 phase →核未决副作用→验数据签名/结果台账→健康探针→决定复用/重跑/人工
质量	validator 失败	回生成阶段重做, 不硬出
预算	token 超	停并总结已得, 不硬跑

- 工具闭合不变量 (**N3**): 每个 tool_use 必须有结果、结果带 ID (tool_use_id); 中断有 cancelled 消息; 取消/权限拒绝 = 正常分支 (blocked), 结果回模型让模型知道”没得到结果”。stopReason=length 的半截参数即使能 parse 也不执行 (pi 教训)。
- 幂等与副作用 (**P0-5 + N14**):operation_id / attempt_id / semantic_input_hash / side_effect_state(prepared|executing|committed|failed) / retry_class / reconcile_action。读取类可重试; 写文件/改数据命令必须幂等键 + 先查结果台账再执行。样本定义冲突不能靠”再问一次模型”。
- **rollback** 边界 (**N13**): 回滚只动”实验历史/上下文”, 不自动撤销文件系统; do-file/结果删除由显式策略决定。
- 健康探针 (**N2**): 压缩/恢复后, 对重建的 Session/ResearchState 跑最小一致性检查 (工具调用闭合、ResearchState 与结果台账一致), 不一致先失败再进 Loop。
- 金句: 超时是”不知道成功没”, 不是”肯定没成功”; 兜底是每类失败有明确降级 + 幂等恢复 + 恢复后先自检。
- M0-M3 不引入 Temporal/DBOS; 迁移触发: 跨天等待/多人审批/分布式 worker/外部不可幂等副作用/明确 SLA (P2)。

4.9.4 评测 (实证 agent 独有的优势: 有 ground truth)

五层评测:

层级	评什么	判分方法
L0 契约	schema、ID、权限、事件一致性、幂等、工具闭合不变量、取消验收、非法事件序列被拒	确定性测试
L1 组件	检索、变量映射、工具参数、skill 触发	gold set + exact/graded
L2 轨迹	走对阶段、不越权、无无效循环、健康探针通过	trace grader + 规则
L3 结果	表格/样本/系数/SE/N/显著性	单元格级数值评分 (每格独立容差)
L4 研究质量	解释是否被证据支持、局限是否充分、探索/确认是否干净 (自由度账本)	盲评专家 + 校准过的 LLM judge

复现基准: ≥ 3 篇异质设计 (面板 FE / IV-事件研究 / 离散选择), 每篇定目标单元格/样本/变量/估计器/FE/聚类/权重/缺失处理; 每指标单独容差 (N/模型结构/数据签名 exact); 系数符号/点估计/SE/p 档位/ R^2 分计; ≥ 3 次独立运行报 pass@1/均值/最差/token/费用。防污染: 遮蔽数字、盲测不给 do-file、无计算 baseline、held-out、正文数字必须回链本

次工具结果。对抗集：小数点/符号篡改、真实引文支持错误主张、虚假引文、PDF injection、恶意变量标签、陈旧记忆、工具结果篡改、重复回放、超时后不确定状态、半截工具调用、取消后模型假装已执行、探索性结果当作确认性（**confirmatory lock** 需拦下）。- 金句：评”怎么得到”，不只评”像不像答案”；能确定性判分的地方用代码判，*LLM judge* 只评开放维度且先与专家校准。

4.9.5 方法学护栏（探索性 vs 确认性，v0.5 N12）

- 显式 **intent**：每次试算标注 `analysis_intent` $\in$ {exploratory, confirmatory}。
- **confirmatory lock**：确认性检验一旦锁定（样本/识别策略/spec），看过结果后改动必须走 **amendment** 记录（谁改/何时/依据/影响哪些已出结果），防止把探索当确认（p-hacking 防线）。
- 研究者自由度账本：记录”什么时候选过哪些建模自由度、当时有没有看过结果”——诚实披露，L4 与写作”局限性”章节使用。
- 实现为可配置策略而非硬规则（学者可关），但关闭要留痕。

4.10 能力边界：外部世界（联网）与输入模态（图片/OCR）（2026-09-07 审计决策）

审计结论：design/audit-stata-practice.md §1/§3。两能力都做成受控，方向已定、落点待 DD-04(权限)/DD-03(上下文)。

联网（受控，非默认开）：- 新增工具 `web_search` / `web_fetch` / `download`；用途先锁：复现包/公开数据、在线补文献、命令/文档核对，不做自由 **browsing**。- 门控随隐私三档：`local_strict` 全禁；`approved_remote` / `mixed_sanitized` 允许，但下载内容一律标 `retrieved_untrusted`（HTML/PDF/zip 都可能注入），落盘走 **artifact** 链（来源 URL + 内容 hash + 版本）。- “下载即自动执行”禁止；复现用作者 **do-file** 属最高风险执行（见 §4.9.2 工具副作用分级 + DD-04）。- 远程搜索会把查询词发给第三方→只在非 `local_strict` 下可用；研究数据/文本不回传（与 `privacy.mode` 绑定，跨边界是授权事件）。

图片 / 视觉输入（分两层）：- 输入层：学者贴图（论文表截图/数据字典/想复现的目标表/扫描件）= `retrieved_untrusted`（图内文字也可能夹注入）；`ModelCapabilityProfile` 增 `vision: bool` (§4.2)；上下文投影层按模型能力裁剪（无 `vision` 的 provider 走本地 OCR/视觉或明确不支持）。- 证据层：`Stata` 自产图（`event-study`/`margins`/`coefplot`/`placebo`/`balance`）= **artifact** → `figure EvidenceCard` (DD-01 §2.6) → `writer` 嵌 `Word` + `caption` 引 `claim`；图可复现（有对应 `dofile`）。读自产图确认”图讲的是 **claim** 讲的事”依赖 **vision**。- 中文扫描 PDF OCR 与贴图解析共用同一视觉能力；`local_strict` 下视觉必须在本地（本地 OCR/本地视觉模型），不许为了看图而把数据图发远端。- `eval/mock` 支持 `image fixture`（防评测退化文本化）。

落地归属：联网→DD-04 工具/权限；图片读入与上下文投影→DD-03；`figure` 证据已在 DD-01。

5. 分阶段路线（v0.4 合并 P1/P2；v0.5 并入 C 项）

- 阶段 0（可选前置，N16）：无 Agent 可信实验内核——Spec → Run → manifest → Validator → 不可变提交 → 重跑，全无 LLM。若求稳先做它再挂模型；与 M0 取舍见 §7。
- M0 骨架：阶段状态机（最小）+ PydanticAI 协议层 + LLM 聚合（`deepseek`+`mock`）+ **SQLite events** 表（**schema**/版本/幂等/可重建字段/**reconcile** 骨架）+ `ResearchState` 最小版 + 物化视图 + 工作区脚手架 + CLI 对话（都进事件表；不存 `thought`）。P1: `ModelCapabilityProfile` + 契约测试骨架。验收含：工具闭合不变量/取消用例（L0）

- **M1** 可行性: file_reader + 双 RAG 落库 (版本链/稳定 chunk_id/source_role) + 混合检索 + 50-100 条 gold set 消融 + 对话式可行性 (证据等级 L-C) + 隐私三档默认 local_strict + OTel 埋点; P1 收尾: stata-mcp 10 工具策略映射/错误分类、抽取 QA。验收: 真实 idea 跑通可行性
- **M2** 实证: 连 stata-mcp + skills(先 panel_id) + approval 门控事件 + 恢复协议 (幂等 + 未决副作用核对 + 健康探针) + **ExperimentFamily** 全成员记录 + 分支语义 + 研究闸门事件。验收: 复现 1-2 篇基准主表 (**L3** 单元格计分开跑)
- **M3** 初稿: Claim/EvidenceCard + evidence_builder + writer + validator(数字/引用/复现/分级证据门槛 L-D) + Word/rtf + 顶刊文风 (style_only 隔离)。验收: 对抗注入 validator 全抓 + 复现集扩 ≥ 3 篇异质 + **L0-L3** 全套
- **M4** 观测/UI: 聊天 + 进度 + trace 回放 (只读物化视图) + OTel 接现成后端。
- **P2** (评测证明需要后再做): 多 Agent 文献广搜 / Qdrant / Temporal-DBOS / 自研 Dashboard / 双投影完整实现 (N15)。

6. 给 codex 的默认建议 (可按需推翻)

- 语言: Python 3.12; 核心依赖: pydantic-ai, sqlite3/aiosqlite、官方 mcp sdk; RAG: chromadb + pymupdf + embedding 接口实现一个; OTel SDK (可选后端)
- **events** 表 (**SQLite**) 是唯一真相源; 物化视图出当前状态 (含 ResearchState/ExperimentFamily); jsonl 仅导出
- 深文件夹深度的东西 (Skill 触发、RAG 检索、writer 分段、阶段机参数) 都做成数据/配置驱动
- 与 stata-mcp 分工: agent 管”研究逻辑/ResearchState/台账/文献/写作”, stata-mcp 管”Stata 执行/结构化结果/溯源”; Stata 细节不渗进 agent
- 外部输入一律不可信 (§4.9.2); 不存思维链, 存可审计决策 (§4.4); 模型想法 ≠ 统计事实, 写入权分离 (§4.8)
- 每步落事件、幂等、可续跑、恢复后健康探针 (§4.9.3); 分级证据门槛不硬出 (§4.8); 评测以复现为北标 (§4.9.4)

7. 待深化 / 待试的开放点 (v0.5 更新)

详细设计进展 (DD-01...07 已全部产出, M0 实现依此): **DD-01** 领域对象与事件账本 (dd-01-domain-events.md, 数据契约) · **DD-02** 阶段状态机 + **harness** (dd-02-phase-machine-harness.md, 控制流) · **DD-03** 上下文/投影/记忆 (dd-03-context-memory.md) · **DD-04** 工具契约与权限 (dd-04-tool-permission.md, 含 stata-mcp 10 工具真实策略映射) · **DD-05 Writer+Validator** (dd-05-writer-validator.md, esttab 表语义/图/Table1/Word) · **DD-06** 评测 **L0-L4** (dd-06-eval.md, 复现 benchmark) · **DD-07** 文献管线 + **skill** 规范 (dd-07-rag-skills.md)。§7 多数待深化点已落到这些文档。**2026-09-07 Stata** 实证审计: design/audit-stata-practice.md。S1 (prep/pipeline、env_sig、表语义 locator/figure 卡) 已修进 DD-01; S2 门控探索环已修进 DD-02; skill requires 已修进 §4.6; 联网/图片 = 新能力方向 (§4.10, 已记为决策), 落 DD-03/DD-04。待做: figure 证据接 writer、描述/平衡表模板、esttab 表语义解析器。

v0.4 已关闭: ledger.jsonl vs SQLite 冲突、thought 持久化、本地隐私 vs DeepSeek 矛盾、双库只靠 collection 隔离、validator 事后校验。**v0.5** 新关闭: events 可重建字段/reconcile (N1) → §4.4; 恢复后健康探针 (N2) → §4.9.3; 工具闭合不变量/半截不执行 (N3) → §4.4/§4.9.3; 研究完整性闸门 + 自由度账本 (N5) → §4.9.2; 结果三层契约 (N6) → §4.8; 分级证据门槛 (N7) → §4.8; 上下文投影层 (N8) → §4.9.1; ResearchState 内容层 (N9) → §4.3.1; Claim/

EvidenceCard+ExperimentFamily (N10) → §4.8; 写入权分离 (N11) → §4.8; 探索/确认护栏 (N12) → §4.9.5; 分支语义 + rollback 边界 (N13) → §4.4; writer fence (N14) → §4.4; 阶段 0 内核 (N16) → §5。

待深化 (**M0-M1**): 1. ResearchState / ExperimentFamily / EvidenceCard 的 Pydantic schema 细化 (字段、版本、批准链) 2. 阶段状态机细节: 迁移协议、blocked/validating 态、失败回退语义、approval 事件 payload 3. events reconcile 算法: fingerprint 去重、终态后不确定事件判定、非法序列拒绝 4. 分支/leaf 语义落地: 一次 idea 内多 spec 分支怎么建/存/展示/合并 (parent_id 树查询) 5. 事件 schema 演进: upcaster + 快照; 单写者事务”事件 + 投影”原子化; writer lease/fence 实现 6. ModelCapabilityProfile 字段与 provider 契约测试用例集; deepseek tool-call/结构化输出稳定性 7. 中文 PDF 解析 (扫描版 OCR 与否)、抽取 QA/错页/双栏/bbox 8. Word 产出链路 (pandoc md → docx? python-docx? rtf 表直出?) 9. mock LLM provider replay 协议, 保证 harness 确定性单测 10. OTel 埋点规范 + 与 events 稳定 ID 关联; 隐私默认不采内容

评测待做 (**M2-M3**): 11. 评测集: ≥ 3 篇异质论文 golden; 目标级容差 (N/结构/签名 exact) 12. 五层 L0-L4 落地顺序与工具; 遮蔽答案/held-out/无计算 baseline 自动化; L4 LLM judge 与专家对齐 13. 检索 gold set: 50-100 条 + dense/BM25/hybrid/rerank 消融 (Recall@k/MRR/nDCG/证据充分率/错误引用率/无答案拒答率/P95/每问成本) 14. 方法学护栏评测: 自由度账本一致性、confirmatory lock 是否拦下”探索当确认”、amendment 链完整 15. 上下文压缩策略具体化: 多级投影怎么折叠、何时压缩、从物化视图拉多少 (§4.9.1)

P2 门控 (评测证明需要再做): 多 Agent 文献广搜 (判据: 独立上下文 + 清晰输出契约 + 可隔离副作用)、Qdrant、Temporal/DBOS、自研 Dashboard、双投影完整实现。

DD-01 详细设计：领域对象与事件账本

文档层级：design/SPEC.md v0.5 的下位细化。本文把 SPEC §4.3.1 (ResearchState)、§4.4 (事件账本/物化视图/分支/恢复)、§4.8 (Claim/EvidenceCard/ExperimentFamily) 落到可实现的契约与算法，并吸收借阅笔记 N1/N9/N10/N11/N13/N14/N15。前提术语与缩写以 SPEC v0.5 为准。M0 实现以本文为数据契约依据；本文不含实现时间表（见 §6 落地顺序只给指南）。**2026-09-07** 审计补丁已并入（依据 audit-stata-practice.md S1）：prep/pipeline 纪律 + 从 prepared 快照重建 (A1)、Provenance env_sig/input_data_sig(A2)、表语义 Locator + figure 卡 (B1/C1)、reconcile 写工具须重建 (A1)、事件目录 + repro.manifest_built/env.snapshot(A3)。

0. 设计输入与要回答的问题

输入：SPEC v0.5 §4.3.1/§4.4/§4.8、§7 开放点 #1-5；research/borrow_from_coding_agents.md B1/B3、C1/C2/C3/C5/C6/C7；research/codex_report.md P0-1/P0-5。

本文必须回答：1. canonical（唯一事实源）与 derived（可重建投影）到底怎么切？2. 每个领域对象长什么样、谁有权写、由哪些事件决定（event-sourced reducer）？3. 事件目录完整清单（事件类型 / 写者 / 投影更新 / 是否需 reconcile）？4. events 表 schema（seq/branch/fingerprint/parent）与分支/回滚语义？5. reconcile、恢复、健康探针的具体算法？6. 单写者事务 + writer lease/fence 怎么落地？7. 快照 + upcaster 怎么落地？

1. 第一性：canonical vs derived

规则：一个事实只有一个 **canonical** 出处；其余全是 **derived**，可由 **canonical** 重建，丢了 **derived** 不丢事实。

类别	内容	可重建?
canonical	① events 表 (append-only, 含不可变 provenance/result/claim 记录) ②不可变工件 (do-files、Stata 结构化结果、原始日志、PDF 原件、数据签名清单) ③用户文件/输入④由确定性组件签发后不可变的 EvidenceCard / Claim 记录	否 (丢即丢, 需备份)
derived	物化投影 (ResearchState / ExperimentFamily 视图 / result_index / citation_index / claim_index / approval_log) ·统计·全文/向量检索索引·embedding ·导出 (jsonl/Word) ·UI/trace 视图	是 (从 events + 工件重建)

推论: - **events** 只存小 **payload** + 工件引用, 不存大二进制 (原始 Stata 输出/PDF 进工件存储, events 存路径 + 哈希 + 结构化摘要)。这是”事件账本”与”artifact store”分离的边界。- EvidenceCard/Claim 一旦由 validator/evidence_builder 签名, 作为不可变事实追加进 events (或独立 append-only claim 表按 events 重建)。正文引用只认这些记录 (写入权分离, SPEC N11)。- 物化投影不许被直接修改——改状态 = 追加对应事件。唯一例外: 幂等工具函数内部临时缓存。

2. 领域对象模型

所有领域对象都是 **reducer** 的输出: Projection = fold(events_from(seq0..S), seed)。构造时只依赖事件与工件, 绝不依赖模型输出/UI。

2.1 公共 envelope (所有领域对象的审计外壳)

```
class Audit(BaseModel):
    provenance_event: str    # 本对象最后一次由哪个 event 更新
    last_seq: int           # 折叠到的事件 seq
    created_at / updated_at
```

2.2 ResearchState (内容层研究状态, SPEC §4.3.1)

代表”当前研究在用什么样本/变量/识别策略/结论前提”。压缩/续跑/分支时保留的是它 + 证据索引, 不是对话摘要。

```
class Filter(BaseModel):
    rule: str; version: int; reason: str | None; approved_event: str | None
```

```
class SampleDefinition(BaseModel):
```

```

source_files: list[str]    # 经 data_refs + 路径审计
filters: list[Filter]      # 样本筛选（每条带版本与批准，防静电改样本）
panel_keys: list[str]; time_var: str | None
signature: str            # 由来源文件哈希 + 筛选规则确定

```

```
class VariableRole(BaseModel):
```

```

    role: Literal["dep", "treat", "covariate", "cluster", "weight"]
    code: str              # Stata 变量名
    construct: str | None  # 构造方式（label/代码摘录）
    construct_do: str | None # 溯源到 dofile
    provenance: str        # 由哪个 spec/事件引入

```

```
class Identification(BaseModel):
```

```

    strategy: str          # DID / IV / event-study / ...
    assumptions: list[str]  # 关键假设（可点回 citable_evidence）
    refs: list[str]        # 依据文献 chunk_id

```

```
class ResearchState(BaseModel):
```

```

    idea_id: str
    phase: str
    sample: SampleDefinition
    variables: dict[str, VariableRole]
    identification: Identification
    current_family_id: str | None # → ExperimentFamily
    current_spec_id: str | None  # → ResearchSpec（主 spec）
    evidence_refs: list[str]     # 当前结论依赖的 claim_ids
    confirmatory_lock: Lock | None # 确认性锁定（见 §4.9.5）
    audit: Audit

```

不变量：- 任何 sample/filters/variables/identification 变更都产生一个带 reason/version 的事件；confirmatory lock 生效后看结果再改必须走 **amendment**。- evidence_refs 只许指向 claim.signed / evidence.card_signed 产生过的 ID。

2.3 ResearchSpec（一次可执行的规格 + 输出契约）

模型只能提议 spec (spec.proposed)，冻结需 **deterministic** 组件/编排器；冻结后生成不可变 hash。**Pipeline** 纪律（审计 **A1, 2026-09-07**）：prep 把 raw → prepared dataset (merge/append、panel 设定、变量构造、winsor 等)。估计 run 一律从 **prepared** 快照启动，不依赖 stata 会话里递增的内存.dta（内存顺序可变、非事务——失败重放会双重 append/漂移，MCP reset+replay 只救会话不救内存数据集）。spec.hash 覆盖 sample_sig+prep+model ⇒ 同一 prepared+spec 得同一可复用结果。prepared 快照是 **derived**/缓存（raw+prep do 可重建）；raw 文件与 prep do 才是 canonical。

```
class ResearchSpec(BaseModel):
    spec_id: str
    family_id: str; intent: Literal["exploratory", "confirmatory"]
    variant_of: str | None      # 父变体（换控制变量 = 新变体）
    data_ref: str; sample_sig: str  # 对齐 SampleDefinition.signature（对 raw 定义）
    prep: PrepRef | None        # 审计 A1: raw → prepared 的 do 引用 + prepared 快照
    variable_mapping: dict[str, VariableRole]
    model: dict                 # est_cmd / fe / cluster / weights ...
    output_contract: dict        # 机器层要提取的字段（系数表/SE/N/R²...）
    preflight: list[str]         # 运行前检查项（变量存在/样本非空/依赖 ados 在/...）
    frozen: bool; hash: str      # 冻结即不可变；hash 含 sample_sig+prep.hash
```

2.4 ExperimentFamily（防只记录喜欢的规格，SPEC §4.8）

一次研究里按”尝试家族”分组所有运行——预计划 vs 探索、停止规则、多重检验计数、全成员 Run、主结果选择理由。

```
class RunRef(BaseModel):
    run_id: str; spec_hash: str; status: str
    outcome: str | None; selected: bool = False; selection_reason: str | None

class ExperimentFamily(BaseModel):
    family_id: str; hypothesis_id: str; intent: Literal["exploratory", "confirmatory"]
    created_by: str
    plan: dict | None          # 预计划 specs / stopping rule（可空 = 纯探索）
    members: list[RunRef]      # 全成员，含被否的
    multiplicity: int = 0       # 该假设下跑的规格数（多重检验）
    main_result: MainResult | None  # 由 human/orchestrator 选择
    audit: Audit
```

不变量：multiplicity = len(members)；主结果必须能回答”为何是这个 spec”（selection_reason + approval）。

2.5 Run / RunAttempt（一次不可变执行）

对应 codex 的”attempt 记录”，是溯源链最小单位。

```
class Run(BaseModel):
    run_id: str; spec_id: str; family_id: str
    operation_id: str; attempt_id: int; semantic_input_hash: str
    status: Literal["pending", "running", "succeeded", "failed", "uncertain", "cancelled"]
    side_effect: Literal["read", "write"] # 读类 vs 写文件/数据类
    closure: ToolClosure | None
    provenance: Provenance | None        # 见下：含 input_data_sig / env_sig（审计 A2）
    result: RunResult | None             # 三层出口（见 2.6）
```

- semantic_input_hash = H(spec.hash + 数据签名 + 参数): 重试同一输入优先复用 **committed** 结果。
- **Provenance** 字段 (审计 **A2**): {do_file, command_hash, data_signature, input_data_signature = H(raw 文件集 + prep do + spec.prep), env_sig = H(Stata 版本 + 关键 ado 清单 [reghdfe/esttab/...] + OS), exec_seq}。env_sig 是复现 eval (换机器/换版本跑作者 do) 里”代码错 vs 环境错”的判据, 缺它归因不清。stata-mcp 能吐 (about/which) 就吐, 不能则由上层在 run 前后各取一次。
- side_effect=read 可安全重放; write 必须走 reconcile (§4)。

2.6 结果三层出口 + EvidenceCard + Claim (写入权分离)

每个 Stata 结果保留三层 (codex N6): - 机器层 (schema 只作用于此, 只读): exit_code / N / coef / se / checksum 等结构化字段。- 模型层: 摘要/警告/下一步 (模型可生成, 不可改机器层)。- 工件层: 完整 do-file / 日志 / 环境 (落工件存储, 事件引用)。

```
class EvidenceCard(BaseModel):    # 由 validator 签发 (模型无权写)
    card_id: str; kind: Literal["numeric", "citation", "sample", "model", "figure"] # figure= 审计 C1
    locator: Locator              # 精确到 页码/表号/行列/result_id/chunk_id/span
    value: dict | None            # 机器层关键值 (带精度)
    machine_hash: str             # 原始机器层哈希, 防改
    signed_by: Literal["validator", "evidence_builder"]; verified_at: str
```

Locator: - 数字/表卡 (审计 **B1**: 表语义键, 不按列序数猜) → {run_id, result_id, table_id, panel, row_label, col_group, stat_type(coef|sel|p|N|r2|fe_flag|...)}。row_label 匹配 esttab 输出里的变量标签/FE 行/聚类行; col_group 匹配列分组 (如”处理 × 组 A”)。writer/validator 需要 esttab → cell 语义解析器: 把 esttab/rtf/csv 排版还原成格子语义, 数值按地按语义键比对 (不是位置猜); display 值反查 canonical 值 (round-trip, 含 0.038*** → 0.0376, p<0.01 与星号、FE 行 “Yes/X”)。- 引文卡 → {chunk_id, doc_id, page, section, source_span}; - 样本/模型卡 → {run_id, data_signature, spec_hash, env_sig}; - 图卡 (审计 **C1**) → {figure_id, run_id, kind(event-study|margins|coefplot|placebo|balance|...), x_var, est_point_ref/ci_ref, caption_ref}; 图本身落 artifact(.gph/.png, hash), writer 嵌 Word + caption 引 claim, 复现 = 图的 dofile 可跑。- 描述/平衡表 (Table 1, 审计 **B2**) 复用数字/表卡语义键; writer 有独立模板。

```
class Claim(BaseModel):          # 研究级命题: Run/卡片 → 初稿段落
    claim_id: str; statement: str # 一句话命题 (经提炼, 非原文)
    kind: Literal["effect", "robustness", "mechanism", "heterogeneity", "design"]
    cards: list[str]             # 支撑的 EvidenceCard
    status: Literal["draft", "supported", "retracted"]
    written_by: Literal["evidence_builder", "validator"] # ≠ model
    superseded_by: str | None    # retract 链
```

写入权分离 (**N11**) 硬规则: EvidenceCard 只能由 validator/evidence_builder (确定性代码) 从 Stata 机器层签发; Claim 只能由 evidence_builder 从已签发的卡片合成。模型/agent 只能写 spec.proposed、决策摘要、结论解读, 写不进 card/claim。

2.7 谁写谁读 (写者矩阵)

对象	谁能创建	谁能更新	谁只读
ResearchState	—（投影）	编排器：由事件 fold	agent/UI/评测
ResearchSpec	agent 提议	冻结 =orchestrator/ validator; 锁定 =confirmatory	运行时
Run	orchestrator/运行时	追加 attempt/result（不 可变）	全
EvidenceCard	validator/ evidence_builder	不可变	全
Claim	evidence_builder	retract/supersede（追加 新卡）	全
ExperimentFamily	orchestrator 建档	human 选 main_result	全

3. 事件账本

3.1 表结构（SQLite DDL 草案）

```
CREATE TABLE events (  
  event_id TEXT PRIMARY KEY,  
  idea_id TEXT NOT NULL,  
  seq INTEGER NOT NULL,      -- 单调；账本内全局排序键  
  branch_id TEXT NOT NULL,   -- 分支  
  prev_event_id TEXT,        -- 本分支内上一条 (=parent)  
  phase TEXT,  
  event_type TEXT NOT NULL,  
  schema_version INTEGER NOT NULL DEFAULT 1,  
  actor TEXT NOT NULL,       -- user|agent|orchestrator|validator|stata_mcp|system  
  source TEXT NOT NULL,      -- 内容来源（信任/审计用）  
  correlation_id TEXT, causation_id TEXT,  
  operation_id TEXT, attempt_id INTEGER,  
  fingerprint TEXT,          -- 语义指纹（重试/重放去重）  
  confidence TEXT NOT NULL DEFAULT 'judgment', -- fact|verified|judgment  
  side_effect_state TEXT,     -- prepared|executing|committed|failed|uncertain  
  payload TEXT NOT NULL,     -- JSON  
  created_at INTEGER NOT NULL  
);  
  
CREATE UNIQUE INDEX uq_events_seq ON events(idea_id, seq);  
CREATE UNIQUE INDEX uq_events_fp ON events(idea_id, event_type, fingerprint) WHERE fingerprint IS NOT NULL;  
CREATE INDEX idx_events_branch_leaf ON events(idea_id, branch_id, seq);
```

```
-- 单写者租约 + revision fence
CREATE TABLE writer_lease (
  writer_id TEXT PRIMARY KEY, token TEXT NOT NULL,
  revision INTEGER NOT NULL,          -- 每次提交单调递增
  acquired_at INTEGER, expires_at INTEGER
);

-- 快照（可选独立表/文件）
CREATE TABLE snapshots (
  idea_id TEXT, as_of_seq INTEGER, kind TEXT,  -- kind: full|research_state|...
  blob BLOB NOT NULL, created_at INTEGER,
  PRIMARY KEY (idea_id, as_of_seq, kind)
);
```

- seq 在账本内全局单调（写入顺序即因果顺序近似）；跨分支只保证本分支有序，树靠 prev_event_id/branch_id 表达。
- fingerprint: 重试/重放去重——对 tool_call/ run 类 = semantic_input_hash，对用户消息类可为空。同 (idea,event__type,fingerprint) 已在账本→命中唯一索引，写入被拒或复用（见 §4.2 幂等）。

3.2 事件目录（完整清单）

升级自 SPEC §4.4 的 6 种 kind 示例。每类给：event__type | 写者 | 何时发生 | payload 核心 | 更新哪些投影 | reconcile 关注？未列出的 event__type 默认被 reducer 忽略（向前兼容）。

#	event__type	写者	触发	payload 核心	更新投影	rec.
1	idea.declared	user/agent	建档	问题/贡献/假设	idea_state	-
2	user.message	user	对话	text	thread	-
3	steering	user	打断/换方向	指令类型	thread	-
4	approval.requested	orchestrator	到人工门	what/为何	approval_log	✓
5	approval.granted / .rejected / .deferred	user	人工决定	what/by/ reason	approval_log, re- search_state	✓
6	spec.proposed	agent	提候选	spec 草案	family.pending	-
7	spec.frozen	validator/ orch	冻结不可变	spec.hash	research_state, family	-
8	spec.locked	orchestrator	confirmatory lock	spec_version	research_state	-
9	branch.created	orchestrator	换 spec/放弃 尝试	fork_event, reason	branch/leaf	-

#	event_type	写者	触发	payload 核心	更新投影	rec.
10	run.requested	orchestrator	发起一次执行	spec_id, operation_id, input_hash	run	-
11	tool.call	orchestrator	请求 stata_mcp	tool, args_hash, side_effect	run(executing)	✓
12	tool.result	stata_mcp/ validator	工具完成/取消/失败	status, machine 层摘要, closure 对齐	run(result)	✓
13	run.succeeded / .failed / .uncertain	orchestrator	判定	provenance, side_effect_states	run, result_index	✓
14	evidence.card_signed	validator	校验通过	EvidenceCard	card_index	-
15	claim.signed	evidence_builder	卡合成	Claim	claim_index, re-search_state.evidence_refs	-
16	claim.retracted	evidence_builder	被新卡推翻	claim_id, superseded_by	claim_index	-
17	family.main_result_selected	user/ orchestrator	选主结果	run_id, reason	family, re-search_state	✓
18	amendment.recorded	user/ orchestrator	看结果后改动	what, before/after, impact	research_state, family	✓
19	phase.transition	orchestrator	阶段迁移	from,to,entry/exit 校验	idea_state, re-search_state	-
20	checkpoint.snapshot	orchestrator	阶段尾	as_of_seq	(快照)	-
21	compaction.bounded	orchestrator	上下文压缩	摘要 + 证据索引范围	(投影视图)	-
22	health.probe.pass / .fail	orchestrator/ validator	恢复/压缩后自检	checklist	run?	-
23	budget.limit	orchestrator	token/步数超	已用/剩余	thread	-
24	privacy.mode.changed	user	三档切换/授权	mode, scope	system	✓
25	provider.fallback	orchestrator	降级	from,to,reason	system (审计)	✓

#	event_type	写者	触发	payload 核心	更新投影	rec.
26	system.restored	orchestrator	从快照恢复	snapshot_seq, reconcile 结果	run?	-
27	artifact.stored	orchestrator/ artifact	大工件落盘	path, sha256, kind	(工件索引)	-
28	files.delete_request	user/ orchestrator	显式清理文件	path, run_id 范围	(工件索引)	✓
29	repro.manifest_build	validator/ orchestrator	交付前 (L-D)	manifest: raw → prep →估计 do → 结果 hash 全链	repro	-
30	env.snapshot	orchestrator	复现/换机前	Stata 版本 + 关键 ado+OS	provenance	-

可读语义：run.requested → tool.call → tool.result → run.succeeded/failed/uncertain 是执行链的最小合法形状（闭合不变量，SPEC N3）；缺任一环即状态不完整，health.probe 或恢复时抓出。

3.3 事件不变量（写入即校验）

- 1. **seq** 连续单调：写者拿 revision 分配 seq；重复 seq 拒绝。
- 2. 闭合：每个 tool.call/run.requested 最终有一个 terminal(tool.result 或 run.*);result 带原 tool_use_id/operation_id, 否则模型下一轮会以为动作没发生。
- 3. **fingerprint** 幂等：同 (idea,event_type,fingerprint) 已存在→复用而非重复执行（尤其工具重试）。
- 4. 非法序列拒绝（**reducer** 是守门员）：未 started 就 completed、completed 又 append attempt、branch 上追认不属于该分支前缀的因果、claim 引用不存在的 card——一律抛错，不做静默修复。
- 5. 写入权：按 §2.7/§3.2 写者列校验（card_signed 只接受 validator 身份的 actor）。
- 6. 只 **append**：不允许 UPDATE/DELETE（含投影也不回写 events）。

3.4 分支与 leaf 语义

- 每次”换 spec/换样本/放弃最近尝试” = branch.created：新分支继承 fork 点之前的全部事件（不可变过去），之后只 append 自己的新事件；idea_state.leaf_id 指向当前分支尾。失败/被否尝试留在原分支可查，不覆盖。
- 查询当前状态 = 沿 leaf_id 所在分支的 prev_event_id 链 fold（+ 快照）；跨分支对比 = 公共前缀。
- 回滚（**rollback**）在领域层只做两件事：① branch.created 移到更早 fork 点（隐藏后续尝试但不删除）；②若确实要清 do-file/结果文件，发 files.delete_request（需审批）。二者永不绑定（SPEC N13）：隐藏失败尝试≠磁盘上文件已删。
- fork 后旧分支可以继续并行跑（同一 idea 内多 spec 各占一支，互不污染 ResearchState——这正是多 agent/并行的隔离基础）。

3.5 reconcile（对”不确定”的处置）

触发：恢复/续跑时，扫描存在 side_effect_state='uncertain' 且无后续 run.succeeded/failed 的事件。

场景	判定	动作
read 工具，超时无结果	可安全重试（幂等）	复用 operation_id, attempt+1 重放；或 query 现状确认
write 工具（Stata 写数据/do），超时无结果	可能已提交	查结果台账/工件： semantic_input_hash 已有 committed 结果→标记 committed 复用；否则 reset+ 重跑同 operation_id；仍无法判定→人工确认
审批已发未回	非不确定，是等外部	保留 pending，不重发
上一进程的 commit 与本次 revision 冲突	旧进程写者	fence 拒绝 (§3.6)，以新 lease 为准

输出一个 system.restored 事件记录 reconcile 决定（reuse/rerun/human/abort），供审计。

写工具失败/不确定一律假设内存数据集可能被污染（审计 **A1**）：对 write 型工具的 reconcile/重放不得依赖 stata 会话内的递增.dta，须从 prepared 快照重建再跑；否则重放会把失败 do 的副作用（已 merge/append 的行）再叠一遍。读取类不受此约束。

3.6 单写者事务 + writer lease/fence

- 提交协议（SQLite 事务内原子）：BEGIN → 校验不变式 → INSERT events（取 next seq）→ 更新物化投影（同一事务）→ bump writer_lease.revision → COMMIT。投影与事件同事务保证”事件已落而投影未更新”永不发生。
- 多进程/崩溃恢复 **fence**：写者先 acquire writer_lease (writer_id, token)（带过期）；每次写带 revision；旧进程恢复后若 revision 已前进（被新进程用过）→拒绝其提交（stale write）。长跑 Stata 会话跨多次进程启动，靠 lease 保证只有一个活跃写者。
- SQLite 需 PRAGMA journal_mode=WAL + 单机单 writer 的显式假设（多 writer 迁移触发条件见 SPEC §7/P2）。

3.7 快照 + upcaster + 恢复协议

- 快照：checkpoint.snapshot 事件触发，存 as_of_seq 的投影全集（或研究状态 + 索引）；恢复时先读最近快照，再对 seq > as_of_seq 的事件按序 fold，不扫全史。
- upcaster**：注册 (event_type, schema_version) → (event_type', payload') 迁移函数表；fold 前逐条 upcast 到当前 schema。schema 演进有迁移测试（L0）。
- 恢复顺序（健康探针 = checklist, SPEC N2）：
 - 读最近快照 + 重建 leaf 分支到最新 seq；
 - reconcile**：处理所有 uncertain 且未闭环的写工具 (§3.5) ——先于一切自动继续；
 - 闭合检查：每个 tool.call/run.requested 有 terminal；没有→标记并决定补结果/重放/挂起；

4. 一致性:run.succeeded 必有 provenance;ResearchState.current_spec_id 存在于某 family;main_result (若有) 有 approval;
5. **compaction** 基线: 压缩 boundary 与当前研究状态一致 (引用都在);
6. 全过→写 health.probe.pass →放行自动继续; 任一 fail →进 blocked 等人/回退阶段。

3.8 存储接口 (LedgerStore Protocol)

```
class LedgerStore(Protocol):
    append(idea_id: str, ev: Event, actor=...) -> int      # 返回 seq; 单写者事务 + 投影
    append_many(idea_id, events) -> int                  # 原子批量
    scan(idea_id, *, after_seq=0, branch=None, types=None) -> Iterator[Event]
    snapshot(idea_id, at_seq=None, kind="full") -> Snapshot
    project(idea_id, *, kinds) -> Projections             # research_state/families/indexes...
    fork(idea_id, from_seq: int, reason: str) -> branch_id
    resolve_uncertain(idea_id, op_id, decision, note) -> Event
    upcast(ev: Event) -> Event                          # 注册表驱动
```

reducer 实现为纯函数: ProjectionState = {research_state, families, indexes...} + apply(state, ev) -> state; project() 由 scan → fold → apply 完成 (先快照后增量)。

4. 关键场景走查 (trace 示例)

4.1 一次 spec 试算 (ESTIMATION)

```
spec.proposed(agent, spec A)
approval.requested → approval.granted(user)    # 研究闸门: 主回归前停等
spec.frozen(A.hash)
run.requested(A, op=op1, input_hash=h)
tool.call(stata_run, side_effect=read|write)
tool.result(rc=0, machine[...], 工件引用)      # 闭合
run.succeeded(provenance{do_file,data_signature})
evidence.card_signed(validator, locator=表3-列2) # 机器层只读 → 卡
claim.signed(evidence_builder, 引用 card)
```

4.2 换 spec 即分支

```
branch.created(fork_at=seq_k, reason="换 FE: 双向FE→单向") # 旧 spec 结果留在原分支
spec.proposed(spec B, variant_of=A) → ... (同 4.1)
# family.members 同时看到 A 的 run 与 B 的 run; A 若未选中则只作探索记录
```

4.3 崩溃恢复 (含 uncertain)

```
进程死前: run.requested + tool.call(write, executing)
恢复: 读快照 → 扫描 → tool.call 无 terminal、side_effect_state=uncertain
reconcile: 查工件/result_index 无 committed → reset+重跑(op1, attempt2) → tool.result → run.succeeded
写 system.restored(reconcile=rerun) → health.probe.pass → 放行
```

4.4 上下文压缩 (checkpoint)

compaction.boundary(summary=ResearchState+已确认 Claim 索引, 保留最近 N do 文件原文为 tail)
只 append; 模型下一轮读的是投影 (summary+tail), 原始历史仍在 events 可随时回放

5. 派生投影清单 (derived, 全可重建)

投影	由哪些事件 fold	用途
research_state	idea/spec/branch/run/claim/ family/approval/amendment	模型 prompt、研究状态展示
experiment_families	spec/run/main_result	防选择报告、L4 方法学审计
result_index	run.* + evidence.card_signed	数值接地、写作渲染、复现 L3
citation_index	(RAG) claim/card	引用接地
claim_index	claim.signed/retracted	writer 输入
approval_log	approval.*/amendment	审计、审批续跑
trace/UI 视图	任意	回放/调试
检索/embedding	工件 + 事件	RAG (derived, 坏可重建)

6. 落地顺序指南 (不排期)

- 1. 无 **Agent** 事件内核 (SPEC 阶段 0 精神): events 表 + append 事务 + 不变式校验 + scan + 快照/恢复骨架——用纯脚本/测试驱动, 先不加 LLM。
- 2. 最小 **domain reducer**: ResearchState + run/result + evidence.card_signed/claim.signed 三个 reducer + 单写者 lease/fence。
- 3. 分支与 **reconcile**: fork + leaf + reconcile 决策表 + 健康探针 checklist。
- 4. 事件目录补全: 按 §3.2 补 approvals/amendments/budget/privacy 等运营事件。
- 5. 之后才是接模型 (spec.proposed / agent_step 由 agent 写) 与 stata-mcp (tool.* 由 MCP 适配写)。

7. 开放决策 / TODO (进实现前要钉死)

- 1. seq 全局单调 vs 每 branch 单调 + 合并排序——先按全局单调 (单写者下等价), 多分支并行写时评估。
- 2. fingerprint 精确算法与哪些 event_type 需要它 (先只对 tool.call/run.requested)。
- 3. payload 存 JSON 文本 vs 拆列——先 JSON (SQLite json_extract 查询), 性能不达标再物化热字段。
- 4. upcaster 注册表组织 (模块内 dict vs 声明式文件)。
- 5. 工件存储布局 (outputs/runs/<run_id>/...) 与 hash 校验时机。
- 6. 快照粒度 (full vs research_state only) 与触发点 (每 phase 尾 vs 每 N 事件)。
- 7. 多分支并行的展示/合并 (UI 后置问题, 现在只定数据语义)。

8. 面试叙事（一句话）

所有状态都是事件的投影，所有正文数字都是一张签名卡片的坐标，模型只能提议、不能署名——这就是把”研究过程”变成”可追责的证据链”的机制。

DD-02 详细设计：阶段状态机与 Harness（编排器）

文档层级：design/SPEC.md v0.5 的下位细化。本文把 SPEC §4.3（Harness 与阶段状态机）、§4.9.3（异常/恢复）、§4.9.4（评测 L2 轨迹）、§5（M0-M4）落到可实现的”阶段 × 属性 + 迁移表 + 门控 + 编排循环”。数据/事件契约以 **DD-01**（design/dd-01-domain-events.md）为准。借鉴：codex 学习册”受约束状态转移 + 主循环 run_turn”；claude-code 学习册”query.ts 状态机 + 状态转移表含 blocked + 多条恢复路径”；claw-code 学习册”research_turn 理想循环（validate → load → while not terminal → verify → persist → maybe_compact）”；pi 学习册”双层循环（外层 follow-up / 内层 tool batch）+ 取消不是错误”。

0. 设计输入与要回答的问题

回答：1. 阶段（phase）与运行态（run_status）如何正交？2. 每个阶段的属性是什么（进入/退出/工具/人工门/失败策略）？3. 合法迁移表长什么样（谁有权触发、什么 gate 通过才走）？4. 门控模型：人工门 vs 机器证据门槛（L-C/L-R/L-D）怎么协同？5. 模型能提议什么、编排器决定什么（ActionProposal 契约）？6. 编排主循环（research_turn）怎么跑、两层循环怎么切？7. 预算/停止/中断（steering）怎么落地成事件与状态？8. 健康探针在哪些点触发？9. M0 最小可跑 loop 的最小形态。

1. 两个正交维度：phase × run_status

阶段（phase）= 研究内容走到哪（长期，存 ResearchState/idea_state）。运行态（run_status）= 此刻编排器在干嘛（短期，等谁/在跑/卡住）。

不要把两者混成一个变量（codex/claude-code 的教训：混了就无法表达”ESTIMATION 阶段被审批卡住”这种常态）。

phase ∈ {IDEA, LITERATURE, DESIGN, DATA, ESTIMATION, ROBUSTNESS, WRITING, VALIDATION, DONE}
run_status ∈ {idle, busy, awaiting_user, validating, failed} # blocked 即 awaiting_user

- busy：编排器在内层 loop 中（可能并行 read-only 工具）。
- awaiting_user：卡在人工门/审批/steering（等价于 SPEC 的 blocked，是正常态不是异常）。
- validating：机器验证中（L-R/L-D），验证事件决定去留。
- failed：当前阶段失败策略裁定需回退/停机，见 §6。

- DONE 后存档，不再接受 agent 自动动作（只收用户 reopen）。

2. 阶段属性表（每阶段一份配置，数据驱动）

每个阶段 = 一份声明式配置 PhaseDef (SPEC: 不要写死，做成数据/配置驱动):

```
class PhaseDef(BaseModel):
    phase: str
    purpose: str
    entry_conditions: list[str] # 断言（可评估的检查名，如 data.ready / feasibility.ok）
    allowed_ops: list[str] # 编排器可执行的 op（见 §5）
    allowed_skills: list[str] # 阶段内可激活的 skill
    exit_gate: Gate | None # 本阶段出口：机器门槛 + 人工门
    max_attempts: int
    failure_policy: FailurePolicy
    min_context: list[str] # prompt 至少注入的投影（ResearchState/result_index...
```

		entry_conditions 主要				
phase	目的	(示例)	allowed_ops	exit_gate	max_attempts	failure_policy
IDEA	立题/研究问题/贡献	idea.declared 存在	ask_user, read, rag(文献)	研究问题 + 文献写成 idea.md 且 user 确认	3	重写
LITERATURE	找定位/缺口/方法	idea 稳定	rag(citable), ask_user	给” 已有人做过 X/缺口在 Y” + 引用清单, user 确认	3	细化检索再回
DESIGN	可行性 + 实证设计	文献结论 OK	ask_user, file_reader(inspect), spec.proposed, rag	L-C 证据 (数 据可构造 + 缺 什么清单) → 人工门	5	回 DESIGN 澄清/回 DATA
DATA	数据就位/变量构造	设计过	file_reader, run(read), data_refs 核 对	变量可构造证 据 + 样本签 名, user 确认	3	回 DATA 补 料
ESTIMATION	主回归/机制/异质性跑通	DATA OK	run(write/read), spec.proposed, interpret, evidence 链	L-R 证据 (rc=0+ 结构 化 + 校验过) → 人工门 (主 回归)	10	回 ESTIMATION 调 spec (受 confirmatory lock 约束)

phase	目的	entry_condition主要				
		(示例)	allowed_ops	exit_gate	max_attempts	failure_policy
ROBUSTNESS	稳健性/安慰剂/机制	主回归批准	run(read), spec.proposed(variant), rag	变体家族跑完 user 确认	8	回 ESTIMATION
WRITING	写实证初稿	结果定	rag(style_only) writer, ask_user	初稿草稿 (仅渲染已验证 claim) →人工门	5	回 WRITING
VALIDATION	终验	草稿在	validator(L-D), 复现检查	L-D 通过 (对抗用例全抓)	3	回 WRITING/回 ROBUSTNESS
DONE	存档	L-D 过	(只读)	存档清单完成	-	-

说明: IDEA/LITERATURE/DESIGN/DATA 的顺序在 SPEC v0.5 §2 里是 DESIGN 之后才 DATA; 此处 DATA 单列是”数据就位”门, ESTIMATION 前的变量构造/样本签核查属于 DATA。实际可把 DATA 并入 DESIGN 之后; 迁移表允许多条路径 (见 §3), 不必每个 idea 都走满九宫。

审计 **D1 (2026-09-07)**: 清洗 + 试 spec 是紧循环, 别让门控卡死探索。真实流程 = 快速回归→发现问题→回去改构造→再跑, 反复几十轮、先于任何正式 gate。PhaseDef 加 gate_mode ∈ {explore, formal}: 进 ESTIMATION 初期 / 纯探索默认 explore (轻门控: 机器门槛只查”能读结果/语法对”, 不请求人工门、不上 confirmatory lock、允许 DATA ↔ ESTIMATION 来回), 只有声明主 spec 或转入 ROBUSTNESS 才切 formal (L-R + 人工门 + lock)。confirmatory lock (SPEC §4.9.5) 不得成为探索期默认。

3. 合法迁移表

迁移只由编排器提交 (模型只能请求)。每条边标注触发条件。

前进边 (默认):

IDEA → LITERATURE user 确认 idea.md
LITERATURE → DESIGN 文献结论确认 (引用清单 OK)
DESIGN → DATA L-C 证据出 (可行/缺数据/缺文献 的下一步被 user 采纳)
DATA → ESTIMATION 样本签名+变量可构造 确认
ESTIMATION → ROBUSTNESS L-R 主结果通过 + 人工批准 (研究闸门)
ROBUSTNESS → WRITING 稳健性结论 user 确认
WRITING → VALIDATION 草稿产出 (writer 只渲染已验证 claim)
VALIDATION → DONE L-D 全过 + 存档

回退/重做边 (允许子集, 其余进 failed 走失败策略):

VALIDATION(fail) → WRITING validator L-D 抓出 → 修
VALIDATION(fail • 复现) → ESTIMATION 复现不过=结果不可靠 → 回跑/修 spec
ROBUSTNESS → DESIGN robust 发现识别策略根本问题 → 重设计 (须 user + amendment)
ESTIMATION(fail • data) → DATA 数据/变量问题 → 回数据 (补料或改构造)

任意 → IDEA user steering（彻底换题）→ 新 idea/branch（DD-01 fork）

拦截/兜底：- 越权边（如 ESTIMATION 没 L-R 就要进 WRITING）由 exit_gate 未过→拒绝迁移，留在原阶段。- run_status=failed 时按该 phase 的 failure_policy 决定自动回退边或 awaiting_user。

4. 门控模型：机器门槛 × 人工门 × 研究闸门

每阶段出口 = 机器门槛 (证据等级) AND (有则) 人工门 (approval)。

门	由谁判	依据	事件	不过怎么办
机器证据门槛 L-C/ L-R/L-D (SPEC §4.8)	validator（确定性）	dofile/rc/结构化结果/复现/对抗	evidence.card_signed / claim.signed / run.succeeded...	回本阶段/前阶段重做，不硬出
人工门（设计结论/主回归/稳健性/初稿前）	user	approval 请求（带”帮用户做决定”的上下文）	approval.requested → granted / rejected / deferred	rejected →回重做；deferred → awaiting_user
研究闸门（改样本/识别/FE/聚类/主结果选择）	编排器强制	变更检测（与 ResearchState diff）	spec.proposed(带变更理由) →需 approval 或 amendment.recorded	阻止静默变更；confirmatory lock 生效时看结果后改必须 amendment
隐私/工具闸门（数据安全）	policy（DD-04）	信任标签 + 工具副作用分级	deny/ask/allow	deny → tell 模型；ask → approval

同步规则：机器门槛先过，才请求人工门（人工不背”数值没验证”的锅）；研究闸门在 machine 判定前先拦（人确认的是”要跑这个变体”，不是事后才知道）。

5. 模型 vs 编排器：ActionProposal 契约

模型产出 **ActionProposal**，编排器执行 ops（codex”受约束状态转移”：模型想法≠统计事实；claude-code”生成命令≠执行命令”）。

```
class ActionProposal(BaseModel):
    # 每条 = 一个候选动作；编排器逐个裁决/执行
    acts: list[Act]
    ask_user: bool          # 需要澄清/等决定
    stop_reason: Literal["gate","budget","need_input","done"] | None

class Act(BaseModel):
    act_type: Literal[
        "propose_spec",    # 提 spec（含 research 变更理由）
        "request_run",     # 请求执行（引用已 frozen spec）
```

```

"interpret",      # 解读某 run (结论摘要, 非 claim)
"select_candidate", # 从候选里选 (最后仍需 user/orchestrator 落)
"ask_user",       # 提问
"read_artifact", "rag_search", "inspect_data",
"request_robustness_variant",
"draft_section",  # 写作提议 (只能引用 claim/card)
]
target: dict | None    # 结构化参数
reason: str | None

```

硬约束: - 编排器把 Act 映射到 op 时逐条过 **policy** (权限/研究闸门/信任标签)。- 模型不能发出这些 act: sign_claim / mark_done / delete_file / phase.transition / 直接改 **ResearchState**。- propose_spec 必须携带对 **ResearchState** 的 diff 与理由 (研究闸门输入); **confirmatory lock** 后解读到新结果再改, 须先 amendment.recorded。- 模型说”我完成了” ≠ 完成: 完成 = exit_gate 证据通过 (§4)。

6. 主循环: research_turn (含两层循环)

借鉴 codex run_turn + claw 的 research_turn 理想循环。分两层: - 外层 (会话/**steering** 粒度): 等用户消息或恢复信号; 把 user.message/steering/approval.granted 转成一次内层调用; 可换 phase。- 内层 (一次 **agent turn**): 在当前 **phase** 内循环直到命中 gate/budget/need_input/done。

```

def research_turn(idea, phase_def, store, policy, budget) -> Status:
    """ 内层: 一次 agent turn。外层由事件驱动 (见 §7)。"""
    while True:
        # 1) 恢复/压缩后健康探针 (见 §8); 2) 构建投影上下文 (DD-03 展开)
        ctx = build_context(project(store, idea))    # ResearchState+result_index+skill
        prop = llm.act(ctx, allowed=phase_def.min_context, schema=ActionProposal)
        store.append(agent_step(decision_summary=prop.reasoning, evidence_ids=...))
        if prop.ask_user: approval.requested(...); return AWAIT_USER

    for act in prop.acts:
        verdict = policy.check(phase=idea.phase, act=act, trust=...)
        if verdict is DENY: # 告诉模型为什么, 回下一轮
            store.append(tool.result(closed_to=..., denied_reason=...)); continue
        if verdict is ASK: # 研究闸门/权限升级
            approval.requested(what=act); return AWAIT_USER
    # ALLOW:
    match act.act_type:
        case "propose_spec": spec.frozen(...); research_state(...) # orchestr. 侧
        case "request_run": run.execute(...) # → tool.call/result/run.*
        case "interpret": agent_step(interpretation=...) # 不许 sign claim

```

```

case "read_artifact"|"rag_search"|"inspect_data":
    parallel_readonly(acts)          # §9 顺序写回
case "draft_section": writer.render(verified_only=True)
# 4) 阶段出口 gate
if phase_def.exit_gate.machine:      # L-C/L-R/L-D validator
    status = validate(exit_gate)     # run_status=validating
    if status is FAIL: apply failure_policy(phase_def); continue/return
# 5) 人工门
if phase_def.exit_gate.human:
    approval.requested(...); return AWAIT_USER
# 6) 预算/停止
budget.tick(); if budget.exhausted: store.append(budget.limit); return STOP_SUMMARIZE
if prop.stop_reason == "done" and no pending acts: break
# 阶段完成 → 编排器提交迁移 (DD-01 phase.transition)
store.append(phase.transition(from=..., to=next(phase_def)))
return ADVANCE

```

要点：- 机器证据没过→不请求人工门；gate 决定”去留/回退”，不是模型自己说了算。- 单次 **turn** 内串行执行有副作用 **op**；只读 **op** 可并行但按原序写回 **events** (pi source-order, §9)。- 半截 **act** (**LLM stopReason=length**) 不执行 (SPEC N3)，整轮丢弃进重试计数。

7. 外层：事件驱动的会话与中断 (steering / follow-up / 打断)

外层是一个事件消费者，不是 **while(true)** 聊天：每次有 **user.message** / **steering** / **approval.granted** / **deferred** / **phase gate** 触发 / 恢复信号时，决定”调一次 **research_turn** / 换 **phase** / 终止”。

- **steering** (换方向/换题)：若仍在同一 **idea** →视情况 **branch.created** (DD-01 §3.4)；若彻底换题→结束当前 **idea**，另开 (并发约束：一次一个 **idea** 工作区)。
- 取消 ≠ 错误 (pi)：cancel 时若内层在跑工具→发 **tool.result(cancelled)** (带原 **operation_id**) + **run** 标 **cancelled**；模型下一轮必须知道”没得到结果” (SPEC N3)。
- 审批回模型：**approval.granted/rejected/deferred** 都作为工具结果回给模型 (**rejected** 带理由)，模型据此改动作——审批是状态机的正常分支，不是旁路。
- 预算/健康探针命中时，外层先总结已得 (**decision_summary** + 已确认 **Claim**) 再停，不留半截。

8. 健康探针放置 (触发点)

health_probe 在四处跑 (结果写 **health.probe.pass/fail** 事件)：1. 进程恢复后 (DD-01 §3.7 顺序 2-6 之后)：**reconcile** 未决 + 闭合 + 一致性检查，过才放行自动继续。2. 压缩 **boundary** 后：重建的投影上下文与 **events** 一致 (引用都在、无半截 **tool**)。3. 跨昂贵阶段前 (**DATA** → **ESTIMATION**、**WRITING** → **VALIDATION**)：机器证据链完整才允许进。4. 进入 **confirmatory** 的 **ROBUSTNESS** 前：确认 **ResearchState** 已锁定、无未决探索性改动。

fail 时→ **run_status=failed** + 按 **failure_policy** 回退，而不是带病继续。

9. 并行与顺序规则 (M0-M3 内仍是单 agent)

- 默认：有副作用 op (stata run/write) 串行；同 phase 内只读 op (rag_search/inspect_data/read_artifact) 可小批量并行。
- 写回 events 按模型原始 **act** 顺序，不是完成顺序 (pi: 语义层保输入序)。
- 同一 idea 内多 spec 分支 (DD-01) 允许各自隔离地读，但写 ResearchState 仍只有一个活分支 (leaf)。真正跨分支并行执行 = P2 多 agent 的门槛 (先评测证明)。

10. M0 最小可跑 loop (指南, 不排期)

最小切片 (阶段 0 + M0 前半, 无 LLM 也可跑通一半): 1. LedgerStore + events DDL + append 事务/不变式/scan (DD-01 §3.1/§3.8)。2. PhaseDef 配置 (九阶段表, data-driven) + policy (研究闸门/信任最小实现) + build_context (先只拼 ResearchState+ 最近 events)。3. mock LLM provider (replay 一个 ActionProposal) 驱动 research_turn; CLI: 你说一句 → 一个 turn → 回一句 → 全落 events。4. 验收: 确定性重放——同输入 replay 得同 events 序列 (单测); phase.transition 只由编排器提交 (L0 用例: 模型 proposal 里带 mark_done 被拒)。

11. 开放决策 / TODO

1. PhaseDef 的 entry/exit 条件断言语言 (查重名/简单规则 vs Python 谓词注册表)。
2. 每阶段的 allowed_ops 与 skill 的粒度: 先列白名单枚举, 成熟后抽成策略文件。
3. max_attempts 的计数口径 (模型重试 vs op 重试 vs 阶段重做是否共用)。
4. 人工门请求的”帮用户做决定”上下文格式 (claude-code 学习册建议审批附上下文——细节放 DD-04 权限)。
5. ROBUSTNESS 与 WRITING 之间的”结论选定”是否要独立子门。
6. steering 换题时是 fork 还是 close+new idea (先 close+new; fork 留给”换 spec”)。

12. 面试叙事 (一句话)

阶段管”研究到哪一步”, 运行态管”在等谁”; 模型只能提议、编排器才提交迁移——把”越权”从 prompt 提醒变成不可违反的状态机不变量, 这就是可信实证 agent 的控制面。

DD-03 详细设计：上下文 / 投影 / 记忆

文档层级：design/SPEC.md v0.5 的下位细化。本文把 SPEC §4.9.1（分层记忆/多级投影/压缩）、§4.10（图片/视觉投影裁剪）、§4.2（ModelCapabilityProfile）与 DD-02 的 build_context()、DD-01 的投影/事件落成可实现设计。数据契约以 **DD-01**、控制流以 **DD-02** 为准。借鉴：claude-code 学习册”Context Engineering：上下文是预算系统 + 多级成本压缩 + 两阶段记忆检索”；codex 学习册”Context Builder 生成可用世界模型、for_prompt 按模型能力投影、token budget 作控制输入、compaction= 语义 checkpoint、从 rollout 重建”；pi”会话树/Context 是投影、compaction 追加 summary+tail 不毁历史”；codex 自动记忆（两阶段 + MEMORY.md/memory_summary + usage 裁剪 + 引用块，截断式借鉴）。

0. 设计输入与要回答的问题

回答：1. “规范 (canonical)”与”给模型的投影 (ContextPack)”之间到底怎么切？投影会不会成为第二份事实源？2. 上下文分几层、每层装什么、多频繁刷新、占多少预算？3. token 预算模型长什么样（每步怎么算、auto-compact 阈值怎么触发）？4. 压缩 = 语义 checkpoint 具体压缩什么、保留什么、怎么写事件、怎么恢复？5. 大工具结果怎么”进摘要不进上下文”，需要细节时怎么按需读？6. 记忆四层里，“项目记忆/偏好”与”证据库”怎么分开、长期记忆借鉴 codex 怎么截断？7. 图片/视觉进上下文怎么按模型能力裁剪？

1. 一条主线：canonical / 投影 / prompt 三者不互相污染

- **canonical** (DD-01)：events + 工件 + 用户输入 + 签发的 Card/Claim。永不因某次 **prompt** 而变。
- 投影（物化视图）：ResearchState / 各 index / family 视图。由 events fold 而来，全可重建。
- **ContextPack**：给某一次模型调用的投影视图（按 intent + 模型能力 + 预算裁剪）。每次调用前现生成，不落盘为事实；可能包含摘要、关键原文 tail、图像、工具输出摘要。
- 不变量：ContextPack 只是”读”规范得出的视图；压缩/裁剪只影响下一轮看什么，不影响 events（压缩也以事件追加 compaction.boundary 记录它做了什么）。这保证任何一轮都能被回放、任何结论都能追溯 (RAG-over-own-history)。

canonical(events/工件/Card/Claim) —fold—> 投影(ResearchState/index/...) —build_context—> ContextPack(本次 prompt)
▲只 append（写者矩阵） ▲每次现生成、不入库

2. 上下文分层（ContextPack 的内容结构）

分四层（claude-code 学习册” 稳定规则/研究状态/证据摘要/原始按需”，映射到实证研究）：

层	内容	来源	刷新	预算权重	能否当证据
L1 稳定规则	系统边界/安全规则/信任标签/当前 skill 的方法论文本（去头）	system/配置	会话级	固定小	否
L2 研究状态	ResearchState 摘要 + 当前 phase/gate_mode + Experiment-Family 主结果/已否列表 + 最近 decision_summary	DD-01 投影	每 turn	中	否（约束）
L3 证据摘要	已确认 Claim 列表（一句/条） + 相关 Card 坐标 + 最近 run 摘要	claim_index/result_index	每 turn	视预算	指向 canonical
L4 原始按需	选中文献块、do-file/结果片段、图、用户原文	按需检索/读 artifact	按需	大、受控	是（带 id）

- L1/L2 一般每次都在；L3 只放结论级；L4 只放当前动作真正要用的。
- 工具大结果默认不整段进 L4：见 §5。

3. ContextPack 结构 + build_context 契约

```
class ContextItem(BaseModel):
    layer: Literal["L1", "L2", "L3", "L4"]
    kind: Literal["rule", "state", "claim", "card_ref", "run_summary", "chunk", "do_snippet", "figure", "image", "user_text"]
    text: str | None
    image: bytes | None      # L4 图片（见 §7）
    ref: dict | None        # canonical 定位（event_id/claim_id/card_id/run_id/chunk_id...）
    omission: bool = False  # 被截断标记（模型须知"省略了什么、去哪看全"）
    tokens: int             # 预算会计用
```

```
class ContextPack(BaseModel):
    items: list[ContextItem]
    usage: dict          # 分层预算使用
    world_state: str      # 一句当前世界模型（codex：Context Builder 生成可用世界模型，非塞满）

def build_context(*, idea, projections, intent, capability, budget, store) -> ContextPack:
    """DD-02 research_turn 调用。纯函数：只读 canonical/投影，不写 events。"""
    pack = ContextPack(items=[])
    pack += rules_for(current_skill, phase, privacy_mode)      # L1
    pack += summarize_research_state(projections.research_state) # L2（只放结论前提）
    pack += summarize_claims(projections.claim_index, top=...)  # L3
    pack += recent_context_tail(store, window=tail_tokens)      # L4 最近原文/do（tail 保精度）
    for want in intent.l4_wants:                                # 当前动作要的细节
        pack += read_by_ref(store, want, capability)            # L4 按需读（文献块/结果/图）
    pack = trim_to_capability(pack, capability)                 # §7 视觉/模态裁剪
    pack = meter_budget(pack, budget)                           # §4 超预算 → 触发压缩信号
    return pack
```

- “world_state” 一句话：由 L2+ 当前 action intent 生成，帮助模型知道此刻在验证什么假设（避免读一堆不相关历史自己猜）。
- 顺序决定质量：先业务投影（中文/证据语义）→后协议编码（供应商差异只在编码层，不反向渗入 canonical, codex 教训）。

4. token 预算模型（预算 = 每步决策输入，不是报错才看）

每步维护：total_budget(per provider/max_context) - model_output_reserve - compaction_buffer = usable_in。

分量	含义	默认（可配）
model_limit	provider max_context	取 ModelCapabilityProfile
output_reserve	给本次生成的余量	~0.25×
compaction_buffer	auto-compact 提前触发区	~0.15×
tail_budget	L4 tail 原文上限	见 §6
per-layer cap	L1/L2/L3 上限	由意图决定

- **meter**: build_context 后统计各层 token; 若超 usable →先降 L4(tail 缩窗/裁剪)→仍超再降 L3 →触发 **compact** 信号 (§6)。绝不让”拼出来的 prompt” 超过 model_limit。
- 会计真实: token 估算按 provider 分词器近似; 错误预算（高估 10% 安全边际）。
- 预算命中写 budget.limit（DD-01 事件目录 #23），外层总结已得再停（DD-02 §7）。

5. 大工具结果：摘要进上下文、完整进账本、细节按需读

每个大结果（Stata 回归输出/日志、大 chunk）按 结果三层出口（DD-01 §2.6 + SPEC §4.8）处理：- 机器层→事件/result_index (canonical)。- 模型层摘要→进 **L3/L4** 的那一行（几行：命令 + 返回码 + 样本量 + 关键系数 + SE + CI + warning + 原始路径 + “要复核读什么”）。- 工件层→artifact（完整 do-file/日志）。

def summarize_tool_result(result) -> ContextItem: # 示例投影格式

```
return ContextItem(layer="L4", kind="run_summary",
    text=("reghdfe y x ctrl, fe cluster(id)\nrc=0 N=12,431 ...\\n"
        "coef=0.038*** (SE 0.012, 95%CI ...) R2=0.31\\n"
        "warning: ... | full: artifacts/runs/r_9a2/log.txt | 复核: ..."),
    ref={"result_id": result.id, "omission_marker": True})
```

- 模型需要更多（如看某行标准误矩阵）→发”读 artifact” act（L4 按需读），而不是把 50 万行塞 prompt。
- 省略要有标记：ContextItem.omission=True + 指向全量路径，模型知道”省略了什么、去哪看全”（codex 的 omission marker；SPEC N3 精神一致——不许假装看到了没给的）。

6. 压缩 = 语义 checkpoint（不是删消息）

触发：meter 超预算 / 阶段尾 / 进程恢复需要更短基线。

压缩产物（一进一出，都写 **events**）：- 进：compaction.boundary 事件，payload = { range:[seq_from,seq_to], summary, claim_ids_kept, artifact_tail_refs, next_allowed_ops }。- 出：下次 build_context 的 L2/L3 用压缩后的 summary（ResearchState + 已确认 Claim 索引 + 最近 tail）替换被压掉的原始。

summary 必须保留因果骨架，不是流水账（codex/claw 教训）：

研究问题 / 数据版本(data_sig) / 已跑 spec 成败(家族+主结果选择理由)
已确认事实(Claim 清单+卡 id) / 被否掉的 spec 与原因 / 识别风险(疑点)
下一允许动作(依 phase/gate_mode)

tail 保留什么：最近 N 条原文（含最近 do/结果行）用于精确参数引用——摘要给方向、tail 给精度，二者缺一不可（pi：追加 summary+tail，不覆盖旧历史）。

不变量：1. 压缩点只能在完整 **turn**/试算之后，绝不截断半条 tool 调用（SPEC N3 / pi 安全切点）。2. summary 里的每个”已确认事实”都必须带 claim/card id，可回 canonical（压缩丢索引 = 丢溯源）。3. 压缩后健康探针（DD-02 §8）：引用都在、无半截 tool、ResearchState 与结果台账一致→过才放行。4. 原事件永不删；压缩只影响”下一轮看什么”。

恢复（压缩后重建）：读到最近的 compaction.boundary →以它的 summary 为 L2/L3 种子→再向前追加 boundary 之后的新事件到 tail →健康探针。参考 codex 从 rollout 的最新方向扫描识别 compaction checkpoint。

7. 图片 / 视觉进上下文（按模型能力裁剪）

- ModelCapabilityProfile.vision ∈ {none, local_ocr, remote_vision}（SPEC §4.2/§4.10）。trim_to_capability 规则：
 - provider 有 vision →图片可直接进 L4（按预算压尺寸/限张数）。

- 无 vision 且允许本地 OCR →图先过本地 OCR/视觉模型，转成文本进 L4（图像本身不入 prompt），OCR 结果标 derived + 源图 artifact id。
- 无 vision 也无本地→该图相关动作降级：只给路径 + “需人工/换 provider”，或拒绝执行（fail-closed，不假装看过图）。
- 图 = **untrusted data**: 学者贴图里的文字不执行；ref 指回 artifact，正文只引用由它签出的 figure 卡（DD-01 §2.6）。
- 自产图（**figure evidence**）: agent 解读自己跑的 event-study/margins 图→读图（vision 或本地视觉）→产解读进 L3，最终由 figure 卡支撑写作（不能凭空说”图显示显著”）。
- **eval/mock**: image fixture 用占位/哈希，测试两分支（有 vision / 无 vision → OCR stub）。

8. 记忆四层落地（“项目记忆/偏好”与”证据库”分开）

重申 SPEC §4.9.1 表格（本设计给它机制）:

层	载体	机制
线程状态	ContextPack（本次）	不落盘；每 turn 重建
项目记忆	研究者档案（偏好/约定/否决记录/样本命名习惯）	约束型，进 L1/L2；跨 idea 复用但仅作约束不作证据
证据库	events + Card/Claim（canonical）	不叫”记忆”，叫事实源；写作只认它
遥测	OTel	运维，非证据

跨 **idea** 长期记忆（研究者档案，借鉴 **codex** 截断式）: - 存 profiles/<researcher>/memory_summary.md + MEMORY.md（条目式：偏好/Reusable knowledge/Failures）+ 分区小文件，git 基线 diff 当”摄入 + 遗忘”信号，usage 裁剪（按最近使用），引用块让”用了哪条”可程序化。- 本地约束: codex 的 phase1 抽取要联网给模型——我们 local_strict 下抽取只能走本地推理/纯规则；mixed_sanitized 下可把脱敏摘要（非研究数据）发给远端。研究数据/证据绝不做进偏好记忆（证据库走 events，不在档案里）。- 范围默认 = 一次 idea（并发约束）；研究者档案可选跨 idea，属 future（P2）。

9. 与 DD-01/DD-02 的接线

- 事件: compaction.boundary（#21）、checkpoint.snapshot（#20）、budget.limit（#23）已存在；本模块是它们的消费者/触发方。
- 对象: ResearchState / claim_index / result_index（DD-01 投影）是 build_context 的输入；DD-01 §5 派生清单对应。
- 控制流: DD-02 research_turn 的 ctx = build_context(...)（DD-02 §6 step 1）在这里实现；PhaseDef.min_context 决定 L1/L2 该放哪些层。

10. M0 最小实现指南

1. summarize_research_state + meter_budget（L1+L2，先不接 L3/L4 裁剪）。

2. 大工具结果三行摘要进 L4 + omission 标记 + “读 artifact” act (mock provider 先支持)。
3. 单层压缩: compaction.boundary 写 summary + tail; 恢复读 boundary 重建 + 健康探针。
4. 记忆档案先不做 (本地 rules 进 L1 即可); 图片分支用 stub。
5. 验收: build_context 确定性 (同输入同 pack); 超预算自动触发 compact; 压缩后健康探针用例 (SPEC §4.9.4 对抗集: 陈旧记忆/引用丢失被抓)。

11. 开放决策 / TODO

1. summary 分层的 token 上限与”压缩时机”默认值; 是否把压缩也做成 gate (每 N turn 强制)。
2. L3 里 claim 的”一句”怎么生成 (evidence_builder 的 claim.statement 已结构化, 直接复用, 不再 LLM 现写)。
3. 图片压尺寸/张数策略 + vision provider 的具体接入 (deepseek 无 vision 时的本地 OCR 选型)。
4. 研究者档案的抽取规则与”偏好 vs 约束”的判定 (避免把学者随口一句当规则)。
5. compaction 跨分支 (DD-01 分支树) 时 summary 该挂在哪个 branch 的 boundary。

12. 面试叙事 (一句话)

我不把历史塞给模型, 而是每次按”意图 + 能力 + 预算”把 canonical 投影成一小包上下文; 超了预算就做语义 checkpoint——摘要留方向、tail 留精度、原事件一条不删。这就是”agent 不是记住, 而是需要时从自己的长期记忆检索”。

DD-04 详细设计：工具契约与权限（Policy）

文档层级：design/SPEC.md v0.5 的下位细化。本文把 SPEC §4.9.2（安全/隐私三档/研究闸门）、§4.10（联网受控）、DD-02 的 `policy.check()`、DD-01 的工具类事件落成可实现设计。数据契约以 **DD-01**、控制流以 **DD-02** 为准。借鉴：claude-code 学习册”Tool Contract: `validate/check_permission/call/summarize_result` 四段 + 权限政策流水线 + 拒绝回模型”；claw-code”Policy/Prompt/Enforcer 三拆 + 固定裁决顺序”；codex”Tool Spec/Registry/Router/Runtime 分层 + Approval 设计”。**stata-mcp** 工具清单核对（C:\Users\user\stata-mcp\src\stata_mcp\tools\, 2026-09-07）：10 工具见 §3，名称/职责已实锤。

0. 设计输入与要回答的问题

回答：工具契约怎么建模？10 个 stata 工具各自的权限策略？`policy` 怎么判、顺序固定？权限维度怎么组合？模型被拒/被打断怎么回？`author do-file` / 联网这类高风险入口怎么控？

1. 工具 = 受控能力对象（不是函数）

每个工具四段分离（claude-code）：**schema** → **permission** → **call** → **summarize**，副产物标注；授权与执行不耦合。

```
class ToolContract(BaseModel):
    name: str
    input_schema: dict          # 模型可见（DD-01 event tool.call 的 args 校验用）
    side_effect: Literal["read","write"] # write= 会改数据/写文件/删文件
    read_only: bool
    destructive: bool          # 删/覆盖原始数据 等
    timeout_ok_retry: bool      # read 类可安全重试
    # 四段：
    validate(args) -> args|err   # schema+ 语义校验（变量名/路径/命令安全）
    permission(ctx) -> PolicyVerdict # 交给 §4 policy（DENY/ASK/ALLOW+ 理由）
    call(args) -> ToolOutcome    # 真实执行（stata 走 MCP / 本地）
    summarize(outcome) -> ContextItem # 三层出口之模型层摘要（DD-03 §5），非整段塞
```

- side_effect 决定 reconcile (DD-01 §3.5): read 超时可重试; write 超时按”可能已提交 + 可能污染内存.dta → 从 prepared 重建”处理。
- 工具调用 = 状态机的正式转移, 不是旁路副作用 (结果带 tool_use_id/operation_id 回模型; closure 不变量, DD-01 §3.3)。

2. 工具分层 (Registry / Router / Runtime)

codex 分层 (模型可见 vs 真实执行分开): - **ToolRegistry**: 全部 ToolContract 注册 (含 schema 与元数据)。- **ActionRouter (orchestrator 侧)**: 把模型 Act (DD-02 §5) 路由到”允许执行的 op”, 决定走哪个工具/哪个 harness 内置动作; 模型只看到 **schema**, 看不到实现。- **Runtime**: 真实执行器——stata 走 MCP client (stata-mcp)、文件/检索走本地实现。加新后端只实现 ToolContract, 主循环不改。

3. 工具清单与策略映射 (含 stata-mcp 10 工具, 已实锤)

工具	职责	来源	副作用	阶段默认	默认策略	参数/前置校验	备注
stata_run	执行一段 Stata 代码	stata-mcp	write (可改内存数据/写文件)	ESTIMATION/DATA	阶段内 allow, restricted 模式	command 过 restrict() (危险命令/路径审计/宏危险); 涉及覆盖原始数据→升级 ASK	核心风险; 代码级受限 见 §6
stata_load_data	载入数据文件到内存	stata-mcp	write (改会话内存)	DATA/ESTIMATION	allow (路径在 allowlist); URL → Deferred/ASK	本地路径审计 (DataPathAuditor); URL 来源未知→ASK + hash	载入后 Research-State.sample 更新
stata_inspect_data	describe/summarize/codebook	stata-mcp	read	全	allow	variables 名校验	只读查看当前数据
stata_data_row	当前数据前 N 行	stata-mcp	read	全	allow	行数上限	agent”看数据”; 结果走三层摘要

工具	职责	来源	副作用	阶段默认	默认策略	参数/前置校验	备注
stata_get_result	读 e()/r() 结构化	stata-mcp	read	ESTIMATION	allow	-	机器层结果入口, validator 依赖
stata_get_help	查官方帮助	stata-mcp	read	全	allow	-	agent 不瞎猜语法
stata_session_history	会话命令日志	stata-mcp	read	恢复/审计	allow(own session)	-	重放/追溯用
stata_break	打断正在执行的命令	stata-mcp	write(控制)	全	allow (对自有会话)	session 归属校验	打断后 rc=1, 引擎存活
stata_task_status	后台任务状态	stata-mcp	read	全	allow	job 归属	异步 run 的配套
stata_export_graphs	导出图为文件	stata-mcp	write(写图)	ESTIMATION	allow (写 _mcp_graphs/)	路径在白名单内	图→ figure artifact (DD-01)
file_reader	读 dta/ csv/xlsx 结构/样本	本地	read	DATA/ DESIGN	allow	路径审计	学者指的数据位置
rag_search	混合检索双库	本地	read	LITERATURE	allow	roles 限制 (style_only 不可当证据)	返回 chunk 带 source_role
artifact_read	读工件/ do/日志/图	本地	read	全	allow	路径白名单	L4 按需读 (DD-03 §5)
web_search / web_fetch / download	联网查复现包/文献/文档 (新增, §4.10)	受控	read (fetch) / down- load=write	视 mode	仅 approved_remote_mixed_sanitized_lo-cal_strict=DENY	下载内容 re-trieved_untrusted来源/版本 artifact; DENY 下载即执行”	用途先锁: 复现包/公开数据/在线文献/命令核对
approval (非工具)	人工门	harness	-	全	-	-	approval.requested/ granted/ rejected (DD-01 #4/#5)

模型可见工具 = 上表 (file/rag/artifact/stata_*/web); **writer/validator/evidence_builder** 不是模型工具, 是 orchestrator 侧的确定性组件 (模型只能 proposal/act, 写不进 card/claim—DD-01 §2.7)。

4. Policy 裁决：三拆 + 固定顺序

借鉴 claw 三拆 (Policy 裁决 / Enforcer 无交互快速 / Prompter 询问), 对应我们: - **Policy**: 完整裁决, 输出 DENY | ASK | ALLOW + reason(可审计)。- **Enforcer**: 热路径, 命中已缓存规则 (如” read 白名单”) 直接 allow/deny, 不打断。- **Approval/Prompter**: 需要人工时, 生成带” 帮用户做决定” 上下文的 approval 请求 (DD-02 §4)。

固定裁决顺序 (每步都可短路并给可理解原因, claude-code/claw 一致):

- 1 无条件拒绝 —— destructive 于白名单外 / 凭据 / 跨 idea 写; DENY(不可协商)
- 2 策略规则 —— (phase × tool × args) 查表: read 白名单→ALLOW_FAST; 越权→DENY
- 3 隐私模式门 —— local_strict 下 web/远程=DENY; approved/mixed 按 §4.10
- 4 研究闸门 —— spec/样本/识别策略/FE/聚类/主结果选择 变更: 默认 ASK (带 diff+理由) 或(explore 期可 ALLOW 但必记 amendment)
- 5 approval ask —— 人工门/高影响写: ASK → approval.requested
- 6 ALLOW

- 每个 DENY/ASK 都回给模型 (结果带 reason), 不是静默失败 (DD-02 §6: deny → tell; ask → 等待)。
- 结果一律审计可解释: 谁判的、依据哪条规则/模式/闸门。

5. 权限维度矩阵 (组合而非开关)

一次 check(act) 实际是 4 维组合: 信任标签 (内容来源) × 阶段 × 工具副作用 × 隐私模式, 再叠加研究闸门与授权来源:

维度	取值	说明
信任标签	trusted_system / user_instruction / retrieved_untrusted / tool_result_verified	检索到的论文即使写” 运行 shell” 也只是文本 span (§4.9.2)
阶段	当前 phase + gate_mode(explore/ formal)	越权工具 (如 WRITING 期 stata_run) DENY
副作用	read / write / destructive / control	决定 reconcile 与审批级别
隐私模式	local_strict / approved_remote / mixed_sanitized	决定能否碰网络/远端模型内容
研究闸门	变更检测 (ResearchState diff)	静默改样本 = 研究事故, 须拦截/留痕
授权来源	user approval 记录	审批是持久化事件, 可查” 谁批了为什么”

权限不是开关, 是带来源与时效的能力: 一个 allow 只对 (阶段, 工具, 参数模式, 隐私模式, 当前 spec) 有效; 换 spec/换 phase/跨 privacy 边界都重新裁决。

6. 高风险入口（作者 **do-file** / 联网复现 / 长命令）

- **author do-file**（复现 **eval** 用）= 外部不可信代码，属 **destructive** 级 + 独立沙箱：只在 **restricted** + 显式审批 + 独立工作目录跑，绝不碰学者真实数据/工作目录；下载的复现包标 **retrieved_untrusted** (§4.10)。
- **stata_run** 长命令：默认 **restricted** 解析（危险命令/路径/宏危险拦截，**guard/restrict.py** 已给）。确需 **unrestricted**（作者原样 **do**）→走第 1 步无条件拒绝的分支改为单次强审批 + 沙箱 + 全量审计，默认关闭。
- 联网：**web_search** 的 **query** 会被发给第三方→只查元信息；**download** 落 **artifact**（URL+hash+ 版本）；绝不把研究数据文本放进 **web query**。

7. 接线与事件

- 每次裁决写审计字段（谁判/依据），执行走 **DD-01** 事件链：**tool.call** → **tool.result(rc/cancelled/denied_reason)** → **run.***；**closure** 不变量保证模型下一轮知道”被拒/被打断/没结果”。
- **approvals/amendment/研究闸门相关事件**（**DD-01 #4/#5/#18**）由本模块触发。
- **DD-02 research_turn** 的 **policy.check(...)**（**DD-02 §6**）在本模块实现；**DD-03** 的 **L1** 规则层注入”信任标签/模式”给模型，让它知道哪些被禁、为何。

8. M0 最小实现指南

1. **ToolContract + Registry**（先 **file_reader** / **rag_search** / **stata_run** / **stata_inspect_data** 4 个 + **approval** 假）。
2. **Policy** 五步顺序的纯函数实现（**read** 白名单 / 阶段 **allowlist** / **research** 闸门 **stub** / **privacy** 常量）。
3. **stata_run** 接 **MCP client**：**restricted** 模式 + **validate()**（**guard** 复用）；**closure** + **summarize()**。
4. 验收（**L0**）：模型 **act** 越权工具被 **DENY** 且 **reason** 回模型；**mark_done/sign_claim** 类 **act** 不存在于 **schema**；跨 **privacy** 边界 **fallback** 触发新 **approval** 事件。

9. 开放决策 / TODO

1. **stata-mcp** 各工具的 **schema** 字段与 **args** 校验细则（接 **MCP** 实例后填）。
2. “研究闸门”触发面的精确 **diff** 检测（哪些字段变才算：**sample.filters** / **variables.role** / **identification.strategy** / **spec.model**）与 **explore** 期自动放行规则。
3. **web** 工具的 **provider** 与去重/缓存（同 **query** 复用上次 **artifact**，省网络 + 审计）。
4. **author do-file** 沙箱目录策略与”允许的重新授权”一次性强审批 **UX**。
5. 是否把 **stata_run** 的 **restricted** 判定结果透出给上层（限制点 vs 已放行点），供 **trace/评测** 断言。

10. 面试叙事（一句话）

工具不是函数，是带 **schema**、副作用、权限、摘要四段的受控能力；每次调用都是信任 × 阶段 × 隐私 × 研究闸门的组合裁决，被拒不是异常而是回给模型的正常结果——权限是带来源与时效的能力，不是布尔开关。

DD-05 详细设计: Writer + Validator (证据→实证初稿)

文档层级: design/SPEC.md v0.5 的下位细化。本文把 SPEC §4.7 (Writer)、§4.8 (EvidenceBundle/Claim/EvidenceCard/分级证据门槛)、§4.9.4 (validator 相关) 与审计 B1/B2/C1 (esttab 表语义、描述/平衡表、figure 嵌入) 落到可实现设计。数据契约以 **DD-01** (Claim/EvidenceCard/Locator)、上下文以 **DD-03** 为准。目标产品: 中文 Word 实证初稿, 数字/引用全可溯源、文风学顶刊、学者能在 **Word** 里继续编辑。

0. 设计输入与要回答的问题

回答: 证据怎么变成段落/表/图? esttab 表语义解析器怎么做 (渲染 + 反查双向)? Validator 查什么、分级证据门槛 L-C/L-R/L-D 的检查项? 图/描述/平衡表怎么进稿? Word 产出链路选什么?

1. 证据→初稿的管线 (写入权分离落地)

Stata 结果 → EvidenceCard(validator 签发, 含 locator) → Claim(evidence_builder 合成)

→ 写作单元(NarrativeUnit) → 初稿(Word)

writer 只消费已签 Claim/Card; 模型只做“组织与措辞”

- **NarrativeUnit**: 最小写作单元 = 一个主张 + 其支撑 (claim_id / 段落文本 / 引用表格或图的 table_id|figure_id / 引文 citation_claims)。整篇 = 有序 NarrativeUnit 树 (方法/结果/解读/稳健性)。
- **render_policy** 硬规则: 模型产出的草稿里任何统计数字、任何表格单元格、任何引文, 必须有对应已签 **Claim/Card** 的 id; 没有 → writer 在渲染期拒绝该句并回查。模型能“提议叙述”, 不能“发明事实” (DD-01 §2.7)。

2. 表格 = esttab 语义对象 (渲染与反查共用同一模型)

审计 B1: 格子身份是 (table, panel, row_label, col_group, stat_type), 不是列序号。定义 **TableModel** 作为中间表示 (esttab → TableModel → rtf/Word; 反查 = 从稿中格值 → TableModel → 原 result_id):

```
class TableModel(BaseModel):  
    table_id: str; title: str; notes: list[str]
```

```
panels: list[Panel]      # 多 panel (A 面板.../机制分列...)
columns: list[Col]      # 每列带 col_group 标签 (如" 处理 × 组 A")
rows: list[TableRow]    # row_label + per-(col,stat_type) 的值
```

```
class Cell(BaseModel):
    stat_type: Literal["coef","se","p","N","r2","mean","sd","t","fe_flag","..."]
    raw_value: str | None # 展示串 (如 "0.038***" / "Yes" / "-")
    canonical: ValueRef | None # → NumericClaim/card (未接地 = 灰格, 不许交付)
```

- **esttab** 解析器 (双向): 读 esttab/estout 的 rtf/csv/txt →还原成 TableModel (含星号、括号 SE、FE 行 “Yes/X/No”、尾行 N/R²/聚类); 反向把 TableModel 渲染成 Word/rtf 表。解析器是 validator 与 writer 的共同底层 (不是只给 writer)。
- **round-trip** 检查: draft 里 0.038*** → TableModel.cell(...) → canonical 0.0376(p<0.01) 数值 + 显著性一致才过 (防小数点/星号/排版篡改)。显示值/原始值分离: canonical 存原始精度, 显示串存展示精度。

3. 内容产出

表: - 回归表: 表语义键渲染 (系数行、括号 SE、星号、FE/聚类/N/R² 尾行)。模板学顶刊 (style_only)。- 描述/平衡表 (**Table 1**, 审计 **B2**): mean/sd/组间差 +t ——独立模板, 复用 EvidenceCard(kind=numeric) 语义键。

图 (**figure**, 审计 **C1**): Stata 图已落 artifact (DD-01) → figure EvidenceCard (含 x_var/est_point_ref/ci_ref) → writer 嵌入 Word, caption 写”图 N: …“并引支撑 claim。没被 **figure** 卡支撑的图不许进正文结论 (agent 不得空口说”图显示显著”)。

方法/解读文字: NarrativeUnit 由 claim 组织;“为什么用这个模型/样本”引用 ResearchState 的 decision/amendment (DD-01) 与文献 (citation)。

4. Validator 规则集 (写稿前/稿中必跑)

检查	对象	规则	不过
数字接地	每格/每数	命中已签 NumericClaim/ card; round-trip (显示↔ canonical 值 + 显著性)	回写/回 ESTIMATION
引用接地	每条引文	溯源到 citable_evidence 真实块 (chunk_id+span); style_only/background 不可当引文	回检索
复现	每表/每图	对应 dofile 可跑: data_sig+env_sig+command_hash 一致 (DD-01 Provenance)	回 ESTIMATION

检查	对象	规则	不过
样本/模型一致	ResearchState	稿里样本口径/变量/识别与 ResearchState/ amendment 一致（防止探索当确认）	回 VALIDATION 初段
图 claim 一致	figure	caption 主张与 figure 卡一致（读图，DD-03 §7）	回解读
完整性	结构	方法/结果/解读/局限齐全；局限用研究者自由度账本（何时看过结果）	回 WRITING
对抗注入	全稿	幻觉数字/假引用/截断星号/陈旧 Claim 抓出	回 WRITING

分级证据门槛实现（SPEC §4.8 的 L-C/L-R/L-D）:

等级	检查项最小集	放行给谁
L-C 可行性	有据文献 + 变量可构造 + 缺项清单 + 学者确认	DESIGN 出口
L-R 单次运行	rc=0 + 结构化结果 + env_sig/ data_sig + 校验过 + 可复现	ESTIMATION 内 claim 可签
L-D 可交付	每表/图有 dofile 可跑 + 数字/引用全溯源 + 复现 manifest + 对抗全抓 + 结构完整	出 Word

5. Writer 流程（分节，学顶刊）

- 1. 取料：从 DD-01 投影取”被选定用于初稿”的 Claim 集（main_result 那支 + robustness 变体）+ 文风样本（style_only 库，仅表达约束）。
- 2. 分节提案：模型给 NarrativeUnit 树（方法/结果/解读/稳健性/局限），每 unit 挂 claim/card id; writer 校验引用完备。
- 3. 逐段写作：模型措辞，writer 拦截”无 id 的数字/引文”（写一个 hook 在句子级扫）。
- 4. 渲染：表走 esttab → TableModel → Word; 图嵌 figure; 描述/平衡表模板。
- 5. validator (§4) 全绿→出 Word; 任何一格灰（未接地）都不许交付（宁缺勿错）。

引用纪律：只能引 citable_evidence; style_only 只影响”怎么说”不改”说什么”（SPEC §4.5）。

6. Word 产出链路（选型仍开放，给倾向）

目标：学者能继续编辑；表/图嵌入；数字可溯源（必要时旁注/批注隐藏引用）。- 候选 A：结构化→中间 markdown/HTML（带 claim id 属性）→ pandoc → docx（样式少、自动化顺）。- 候选 B：python-docx 直出（样式控制细，但表格/图 + 溯源批注工作量大）。- 候选 C：rtf 表直出 + Word 主文档引用。- 倾向 A + 溯源旁注：NarrativeUnit 渲染成 docx 时把 claim/card id 写成隐藏文字或批注（学者平时不见、评审可见），保” Word 可编辑” + “数字可追”。[TODO] 选型在 M3 前用样例定（拿一篇真稿双跑 A/C 比可编辑性）。

7. 接线

- 输入：DD-01 的 claim_index/card/result_index + ResearchState (main_result 选择)；DD-03 只负责把”要写哪些 claim”放进程；writer 不读对话。
- 事件：产出时追加（写作/交付相关在 DD-01 事件目录补 draft.rendered/draft.validated, 或复用已有；见 §8 TODO）。
- validator 失败回 WRITING/VALIDATION (DD-02 §3 回退边已含)。

8. 开放决策 / TODO

1. esttab 解析器对多种 **esttab** 方言/分隔符/编码 (**GBK**) 的健壮性 (M3 前用真 esttab 输出建 fixture 集)。
2. Word 溯源旁注形式：docx 批注 vs 隐藏文字 vs 单独 reference map (学者可编辑性/评审可查性取舍)。
3. 表/图编号与正文交叉引用管理 (TableModel.figure_id 全局表)。
4. 长稿分节写作的并行 (结果多张表时先方法→表逐个，不并行；稳健性可 batch) ——与 DD-02 并行规则一致。
5. “局限”章节自动收集：自由度账本条目→草拟局限 (学者确认后进稿)。

9. 面试叙事（一句话）

我们把 esttab 表格还原成语义格子、把每格数值接回某次 Stata 结果，写作器只认带 id 的证据——模型能帮忙写字，但一个没来路的数字都进不了 Word。

DD-06 详细设计：评测（五层 L0-L4 + 复现 benchmark）

文档层级：design/SPEC.md v0.5 的下位细化。本文把 SPEC §4.9.4（五层评测、复现基准、对抗集）、DD-01（事件/claim 溯源）、DD-05（validator/表语义）落到可实现评测设计。定位：实证 agent 独有的 ground truth = 复现已发表论文。目标不是”好看”，是能证明”数字对不上就是失败”。

0. 设计输入与要回答的问题

回答：五层每层测什么、判分怎么实现、跑在哪；复现 benchmark 怎么选论文、怎么防污染、容差怎么定；mock/确定性替身怎么做；对抗集怎么自动化。

1. 评测总体（跑什么、在哪跑）

- 每次改动跑：L0（契约/回归）快、每次提交跑；L1-L2（组件/轨迹）在 CI/评测机跑；L3-L4（数值/研究质量）慢、在有 Stata + 黄金数据的环境跑（黄金 do 只读、跑在独立沙箱）。
- 确定性替身：LLM 用 mock provider（录 replay）；Stata 用 fixture 结果/真 stata-mcp（独立目录）；文献用小型 golden PDF 集。评测必须可重放：同一输入→同一 events 序列→同一分（SPEC 金句”评怎么得到”）。

2. L0 契约（每次提交）

确定性单测，测对象 = 系统自己的合同，不用 LLM：- schema/幂等/事件一致性：DD-01 不变式（seq/fingerprint/closure/非法序列拒绝）。- 工具闭合不变量（N3）：每 tool_use 有 terminal、带原 id；半截 act 不执行。- 取消验收：打断后带原 id 的 cancelled 结果、下一轮模型知道没拿到。- 权限（DD-04）：越权 act 被 DENY 且 reason 回模型；sign_claim/mark_done 不在模型 act 集。- 写入权：validator/evidence_builder 之外无人能产出 card/claim。- 恢复：reconcile 决策表各分支、health_probe checklist、writer fence。判分：全过/全挂（硬门槛）。

3. L1 组件（gold set + graded match）

组件	测什么	判分
检索	50-100 条真实研究问题 gold set	Recall@k / MRR / nDCG / 证据充分率 / 错误引用率 / 无答案拒答率 / P95 / 每问成本
变量映射	给自然语言” 公司年龄、托宾 Q… “→ 应构造变量	exact/graded match 对代码
工具参数	场景→应选工具与参数	exact
skill 触发	场景→应激活哪个 skill	exact
表解析	esttab fixture → TableModel (DD-05)	cell 级 match

4. L2 轨迹 (trace grader + 规则)

输入：一条完整任务（真实 idea 或脚本化）跑完→事件流。规则判：- 走对阶段（合法迁移，DD-02 §3）；无越权；无无效循环（同 op 卡死/重复率超阈值→报）。- 不越 **privacy**/研究闸门；approval 只在对应 gate 触发。- 健康探针：恢复/压缩后先自检再进（否则 L2 红）。- 预算：未在耗尽后继续硬跑（budget.limit 后停）。判分：规则 + trace grader（可调 LLM-as-judge 仅评” 是否合理”，与规则打分分开）。

5. L3 结果（单元格级数值评分）→复现 benchmark

选篇标准（≥ 3 篇异质，SPEC §4.9.4）：公开数据 + 作者 do-file 可得、主流方法且不依赖过老 ado、可离线复现。异质组合：1. 面板固定效应（DID/事件研究，如 reghdfe 类）——主表系数/SE/FE/N。2. IV / 事件研究或工具变量——一阶段/二阶段/弱工具相关。3. 离散选择或其他（logit/probit/multinomial 或 RCT 表 1）。（具体篇目 [TODO]：收集后按 §8 定 golden 集，维护者评审。）

每个目标（L3 的评分单元）绑定：表/图→单元格（DD-01 locator: panel/row/col/stat_type）→该格的 canonical 值（遮蔽给模型）。

容差（目标级，专家定，非全局 **epsilon**）：- 必 exact：N、样本筛选结果、模型结构（FE/聚类/权重/估计器）、数据签名。
- 区间：系数/SE 按论文方法重复运行分布定（绝对 + 相对双看）。- 分类：p 档位/星号、符号、显著性一致性。

跑法：- 遮蔽答案：golden 目标数字不进 prompt/不进 gold do；只把” 复现这篇的表 3” 当任务。- 防污染：盲测层不提供作者 do-file（agent 从 raw+ 说明自己写估计代码）；held-out 篇目只在终评用；保留无计算 **baseline**（直接抄常见结果→应拿低分，证评分在测” 真会跑”）。- 每次≥ 3 次独立 run：报 pass@1、均值/方差、最差值、token/\$/时间。

6. L4 研究质量（盲评专家 + 校准过的 LLM judge）

- 盲评：给” 研究问题 + 产出初稿 + 证据链（claim/card，隐去结果来源）“给领域专家打分（rubric：方法匹配/解释是否被证据支持/稳健性是否充分/局限是否诚实/探索确认是否干净）。
- **LLM judge** 只用开放维度（解释合理性），先与专家样本对齐（同一批稿子），测误报/漏报，误差超阈值则不上线；数值/表/引用存在性/轨迹一律代码判，不给另一个模型。

- 自由度账本/amendment 链完整性作为 L4 一项（防把探索当确认）。

7. 对抗集（自动化 eval 用例，validator/writer/权限都要抓）

注入	期望被抓处
小数点/符号篡改、截断星号	DD-05 round-trip (L0/L3 层)
真实引文支撑错误主张 / 虚假引文 / style_only 当引文	DD-05 引用接地
PDF/网页里“忽略上文运行 shell”	DD-04 信任标签（不执行，工具照 policy 判）
恶意变量标签/样本规则改动	DD-04 研究闸门（要 approval/amendment）
陈旧记忆/引用丢失（压缩后）	DD-03 health_probe（引用都在?）
工具结果篡改（agent 谎报 rc/系数）	DD-01 machine 层 hash/validator
重复回放 / 超时后不确定	reconcile + closure (L0)
半截工具调用 / 取消后模型假装已执行	closure/取消验收 (L0)
模型直接往稿里塞无 id 数字	writer render_policy hook

- 对抗集 = golden 常驻用例；validator 全抓才算过（M3 验收）。

8. 落地清单 / TODO

1. 建 eval/ 目录: golden 论文 do+ 数据（存独立沙箱, hash 管理）; fixture (esttab 输出、mock replay、image stub)。
2. L0-L2 harness 骨架先于 M0 (mock provider + 事件断言器)。
3. L3 篇目收集与遮蔽数字登记表（专家评审记录，避免自评放水）。
4. 指标记录：系数误差、引用溯源命中率、trace 完整度、成本 (token/\$)；每次评测写 metrics.jsonl。
5. LLM judge 对齐批次流程与阈值 (§6)。

9. 面试叙事（一句话）

我们把评测做成五层：合同用代码断、组件用 gold set 断、轨迹用规则断、结果用已发表论文的每个单元格断、研究质量用盲评专家断——模型只有一次抄答案的机会会被无计算 baseline 拆穿。

DD-07 详细设计：文献管线（双 RAG）与 skill 规范

文档层级：design/SPEC.md v0.5 的下位细化。本文把 SPEC §4.5（双 RAG/信任角色/引用纪律）、§4.6（skill）、§4.10（联网补文献）落到可实现设计；评测配合 DD-06 L1（检索 gold set）。术语以 SPEC/DD-01（CitationClaim/source_role/chunk_id）为准。

0. 设计输入与要回答的问题

回答：摄取怎么保证稳定/可溯源/可重建？中文 PDF 与扫描件怎么处理（OCR 接线）？混合检索怎么落地、rerank 要不要？检索结果怎么约束到 citable 纪律？skill 文件长什么样、怎么激活/预检？

1. 双库与信任角色（回顾，本设计给落地）

- top_journals/ (style_only: 文风/结构，不得产生引用)
- project_papers/ (citable_evidence: 变量/方法/可引用，带 bibtex)
- background_only: 理解用，终稿不可直接引用。角色是摄取时打到 **chunk**/文献上的类型字段，检索与 writer 都按它过滤（不是靠 prompt 提醒）。

2. 摄取版本链（可溯源、可重建、可重跑）

file_hash → parser_version → page/section/bbox → chunk_id(稳定)
→ source_role → 文本/图 → embedding_version → index_version

- 稳定 **chunk_id**: sha256(doc_id|parser_version|locator|first~last)，内容变了 chunk_id 变，旧 chunk 失效保留历史（不覆盖）——文献更新不污染历史引用（DD-01 事件引用旧 chunk 仍可查）。
- **canonical/derived**: 原始 PDF/文本 = canonical (哈希存储，只读)；分块、embedding、向量索引 = derived (可重建、可重灌)。
- 摄取记账：每次摄取写事件（可复用 DD-01 artifact.stored + 索引清单），失败可重放（幂等：同 file_hash 同 parser 不重复建块）。
- 抽取 QA (审计 A 精神)：每批抽查页序/表格/脚注/双栏/错页；parser 版本升级必须回归 (L1 表解析/抽取 fixture)。

3. 解析管线（含中文/扫描）

1. 文本 PDF: pymupdf/pdfplumber → 页/bbox → 去页眉页脚/脚注归属。
2. 表格/图: detect → 表格单独成块（供 DD-05 esttab 对照/表引用），图/公式成图块（后续可 OCR/视觉）。
3. 扫描件（中文）: 走 OCR（接 DD-03 §7 视觉能力; local_strict 用本地 OCR，禁止因“要解析文献”把文献发远端）。OCR 结果 = derived, 标 ocr:true + 置信度，引用 span 允许但标注为 OCR 转写。
4. 分块策略: 按节/段优先（保语义），块带 doc_id + page/section/bbox + role; 中英混排不硬切。

4. 混合检索 + rerank（落地）

```
class Retriever(Protocol):
```

```
def search(query, *, top_k, roles) -> list[Chunk]: ...
```

- 内部 = 向量召回（Embedder → VectorDB）+ 词法精确（BM25/FTS 于 chunk 文本，grep 于“确定词/变量名/表名”）并查→合并去重→（可选）rerank。
- 角色过滤在检索参数：写作/可行性取证只能 roles=[citable_evidence]；文风查询 roles=[style_only]；二者永不混。
- **rerank** 是否上由 gold set 消融定（DD-06 L1 指标），先不加，数据说话。
- 每块带回 doc_id + source_role + page/section/span；检索结果本身是 untrusted（DD-04 信任标签），不能当指令。

5. citable 纪律（引用溯源）

- 写作引用只能来自 citable_evidence 的块，且建 CitationClaim（chunk_id+span+ 版本）——**A** 文结论不许安到 **B** 文（DD-01 locator）。
- 检索到的候选若被引用，writer/validator 用 CitationClaim 校验存在且支持该句主张；不支持→回检索/拒引。

6. 联网补文献（§4.10 接线，可选）

- web_fetch/download 拉到新 PDF（NBER/期刊开放版）→走同一摄取链（来源 URL + hash + parser 版本），标 retrieved_untrusted →入 citable（需角色/许可判断）或 background。
- 本地没有、确认是开放文献→approved_remote/mixed 才拉；local_strict 禁。
- 拉文献≠拉进证据：下载后是否可引用由“来源角色 + 学者确认”定，不自动升级为证据。

7. skill 规范（文件化方法论 = 版本化政策包）

目录 skills/<slug>/SKILL.md, 用 frontmatter + 正文:

```
---
```

```
name: panel_did
```

```
version: 1.2.0
```

```
role: methodology      # 或 robustness / mechanism / writing
```

```
triggers:               # 命中条件（由 harness 评估当前 task/ResearchState）
```

```

- phase: ESTIMATION
- needs: [did, event-study, parallel-trend]
requires:      # 审计 D2
ados: [reghdfe, esttab, coefplot]
stata_min: "17"
prechecks:
- data.has_panel_keys
- data.has_variable(treat)
steps:        # 有序；每步可带 → evidence/act
- 平行趋势：先事件研究/安慰剂再进 DID
- 主 spec：fe + cluster(id)，过研究闸门
rules:        # 方法学硬规则（禁/允）
- forbid: 单期 DID 无 parallel-trend 证据却宣称因果
- forbid: 把探索性多 spec 只挑显著报
examples:     # 每个给"输入→应产出"可判例
- run a: ...
disabled_when: ...

```

- **skill** = 政策包 (claw 学习册 + SPEC §4.6): 给”什么时候用、先检查什么、步骤、哪些不许”——不是 prompt 片段，是可判的规则 (L1 skill 触发评测)。
- 激活流程: harness (DD-02 §6) 按 phase + triggers 从 Registry 选→ preflight 查 requires (which <ado>, 缺则提示安装/不静默跑) →注入 DD-03 L1 稳定规则层。
- 变更 = 新版本; skill 回归 (跑它的 examples) 在 L1/L2。

8. 目录/接口

```

libraries/top_journals/{pdf, chunk/, index_v/} # role=style_only
libraries/project_papers/{pdf, bibtex.json, chunk/, index_v/} # role=citable_evidence
skills/<slug>/SKILL.md

```

检索接口: Retriever.search(query, top_k, roles, filters); 摄取接口: IngestPipeline.ingest(lib, path, parser_ver) (幂等); skill Registry: skills.load(slug) / match(ctx)。

9. 开放决策 / TODO

1. 中文 PDF 是文本版还是扫描版——先抽样验证，决定 OCR 优先级 (SPEC §7 #7 落地)。
2. embedding/向量库具体 (bge-m3 本地 vs API; Chroma 起步); 在 privacy 三档内定。
3. chunk 大小/重叠与”表格块”切法 (影响引用粒度) ——用 gold set 消融。
4. rerank 上不上、用什么。
5. 文献去重 (同文多版本、工作论文 vs 发表版) 与”失效保留历史”的展示规则。

10. 面试叙事（一句话）

文献不按”篇”存，按”带角色与页码的块”存，写作只能引用 citable 块的原文 span——把”防编造引用”从口号变成检索与渲染的类型约束；skill 是带前置检查和禁用条件的可判政策包，不是会背的 prompt。

设计审计：按 **Stata** 实证研究实际需求回看 **SPEC**

v0.5 / DD-01 / DD-02

审计对象：design/ SPEC.md v0.5、design/ dd-01-domain-events.md、design/ dd-02-phase-machine-harness.md。审计视角：真实实证（社科/计量）研究者在 **Stata** 里到底怎么干活——数据清洗/构造、反复试 spec、出表、出图、复现。发现按”会不会在真实使用时咬我们一口”排序。结论性质：审计 + 修补建议，未自动改动 SPEC/DD；每项给严重度（S1 必修/S2 该修/S3 建议）与修补落点。

0. 一句话结论

方向仍成立，但有三处”真实使用会咬人”的洞：①把”一次回归”当最小可复现单元，低估了长数据 **pipeline** + 环境 (**ado/Stata** 版本) 依赖对可复现的影响；②只把数字当证据，没把表格语义和图形当一等证据；③面向”封闭本地 PDF 库”设计，没有给”外部世界”（联网复现包/公开数据）和”非文本模态”（图/扫描件/用户贴图）明确的受控入口。前两类是修 DD 就能关的洞，第三类是新能力方向，要开决策。

1. 两个能力问题的直接答复

1.1 能不能联网？

现状：设计上不能（除 **provider API** 外全本地）。- SPEC v0.5 只有隐私三档门、provider.fallback、files/网络进工具副作用表里标”联网下载 Deferred”。没有 **agent** 级 **web_search / URL fetch** 工具。- 真实需求：①复现 benchmark 要拿公开论文复现包（作者 do-file + 数据，多数在期刊/NBER/网上）；②可行性对话可能要看新文献（不在本地库）；③找公开数据。- 建议：做成受控能力，不是默认开。新增 **web_search/fetch/download** 工具，归 DD-04 权限表：local_strict 全禁；approved_remote / mixed_sanitized 允许，下载内容一律标 retrieved_untrusted（可能含注入），落盘带来源 URL + 内容 hash + 版本（复用 artifact 链）。绝不允许”下载后自动执行”。作者复现 do-file 属于最高风险执行（见 §2.6）。

1.2 能不能输入图片？

现状：设计上不能（全链路文本假设）。- SPEC/DD 无 modality。真实需求三类：①学者贴图（发表表格截图 / 数据字典 / 论文图 / 想复现的目标表）当输入；② agent 读 **Stata** 自己产出的图（event-study/margins/coefplot/placebo/balance）做解读与写稿、嵌 Word；③中文扫描 PDF 的 OCR。- 建议：接受“图 = 一等外部输入 + 一等证据产物”，但分两层：- 输入层：用户贴图 = retrieved_untrusted（图里也可能夹 prompt 注入文本）；ModelCapabilityProfile 增 vision: bool；上下文投影层按模型能力裁剪（codex for_prompt 已借鉴）；local_strict 下无远端视觉→走本地 OCR/本地视觉模型，或明确该模式不支持贴图解析。- 证据层：Stata 出的图落 artifact (.gph/.png, hash) →新增 figure EvidenceCard → writer 嵌 Word, caption 引 claim；图也要可复现（有对应 dofile）。深度见 §2.3/§2.5。- 中文扫描 PDF OCR 与“贴图解析”共用视觉能力，是同一块。

2. 审计发现（按域）

A. 执行粒度与可复现性（S1：必修，改 DD-01/DD-02）

A1 | “**Run** = 一次回归”粒度太细，真实可复现单元是“长 **pipeline**”。实证中 90% 时间在数据构造：加载 → merge/append → panel 设定 → generate/encode → winsor → 跑回归。若每个 Run 只记“最后一条回归命令”，重放必须能复现整条链。- 修：Run 之上加 Pipeline/DataPrep 语义——ResearchSpec 明确 prep（从 raw → prepared dataset 的脚本/do），每 run 记 input_data_signature = H(raw 文件集 + prep do + Stata/ado 版本)；sample_definition 对 raw 定义、run 对 prepared 定义。优先“每次从 **prepared** 快照重建，不依赖内存增量状态”（Stata 内存数据集是顺序可变、非事务的，重放失败 do 会双重 append/漂移——MCP session reset+replay 只救会话，不救内存数据集污染）。- 落点：DD-01 §2.3 ResearchSpec（加 prep 与 run_from）、§3.5 reconcile（write 判定含“内存数据集污染→必须从 prepared 重建”）。

A2 | **provenance** 缺环境指纹（**Stata** 版本 / **ado** 依赖 / 机器）。结果由 reghdfe/esttab/coefplot 等用户安装 **ado** 决定；不同 Stata 版本/平台可能微差。复现 eval（换机器跑作者 do）没有 env 指纹就说不清”是代码错还是环境错”。- 修：provenance（DD-01 §2.5）加 env_sig = H(Stata 版本 + ado 清单(关键项) + OS)；stata-mcp 层能吐就吐（含 about/which reghdfe），不能则设计约束要求。- 落点：DD-01 §2.5 + §4 场景、M2 复现 eval 容差判断。

A3 | 复现包 (**repro bundle**) 与 **author do-file** 是两类不同风险。- 修：L-D 交付物加 repro_manifest（raw 引用 + prep + 估计 do + 结果哈希 → 一键重跑）；validator L-D 检 manifest。- 作者 do-file（eval 用）是外部不可信代码：只准在 restricted + 显式审批 + 独立 sandbox 目录跑，绝不碰学者真实数据/工作目录；下载的复现包标 retrieved_untrusted。- 落点：DD-04 权限表加“author_dofile = 最高风险”；DD-01 事件目录加 repro.manifest_built。

B. 表格与数字接地（S1：必修，改 DD-01 §2.6/§4.8 + writer/validator 设计）

B1 | “表 3-列 2”的 **locator** 太粗；表格是 **esttab** 语义对象。真实表 = 多 panel、row label、多列、括号 SE、星号、Yes/No/X 的 FE 行、N/R²/聚类尾行。格子的身份是 (table, panel, row_label, column_组, statistic_型 [coef/se/p/N])，不是序列数。- 修：NumericClaim.locator 用表语义键；validator/writer 要有 **esttab** → **rtf/csv** 的解析器（把排版还原成 cell 语义），数字接地按 cell 语义比对，不是按位置猜。排版层值 vs 规范值 round-trip 反查。- 落点：DD-01 §2.6 Locator 扩字段；新 DD（writer/validator）写 esttab 表解析。

B2 | 描述统计/平衡表是独立产物。Table 1 (mean/sd/组间差 +t 检验)、balance test 常见且常进主稿；它们不是”回归系数”。- 修：EvidenceCard.kind 已泛化 numeric 够用，但 writer 要有”描述/平衡表”模板；L3 复现评分要含这类表。落点：writer 模板清单。

B3 | 显示 vs 原始精度已有 (SPEC §4.8)，但补 **round-trip** 测试：draft 里”0.038***” “必须能反查到规范值 0.0376($p<0.01$) 且格式一致。落点：validator 单测 (对抗集已含小数点篡改，加格式层)。

C. 图形作为证据 (S2: 该修；真实使用缺它活不下去)

C1 | 图没有进证据链。Stata 的 event-study/margins/placebo/coefplot/balance 图是正文核心，目前 SPEC 只把”图”当 outputs 附件，writer 只谈表格 + rtf。- 修：图 = artifact (.gph/.png, hash) → figure EvidenceCard (数据点/CI/纵轴含义 locator) → writer 嵌 Word + caption 引 claim → 复现 = 图的 dofile 可跑。这依赖 **vision** 能力 (§1.2) 读图确认”图讲的是 claim 讲的事”。- 落点：DD-01 证据链加 figure 卡；DD-03 上下文含图；writer 模板加图。

D. workflow形态 (S2: 该修，改 DD-02)

D1 | “清洗 + 试 spec”是紧循环，不是线性门。真实流程：跑个快速回归→发现变量问题→回去改构造→再跑，反复几十轮，先于任何正式 gate。若 DESIGN/DATA/ESTIMATION 门控过严、或 confirmatory lock 过早，会把探索卡死。- 修：门控严格度做成每阶段可配置；进 ESTIMATION 初期默认”轻量探索环” (不请求人工门、不上锁)，只有声明主 spec / 转 **ROBUSTNESS** 才收紧 (DD-02 §2 已有 PhaseDef 数据驱动，明确调这个旋钮)。exploratory/confirmatory 显式标注已做 (SPEC §4.9.5)，别让 lock 变成探索期的默认。

D2 | **skill** 要声明依赖 (ado/Stata 版本)。panel_did 依赖 reghdfe、esttab、coefplot 等；机器没装就跑不过，且会污染”代码 vs 环境”归因。- 修：skill 文件加 requires: {ados:[], stata_min:”...”}；skill 激活时 preflight 检查已安装 (which reghdfe)，缺则先建议装/提示，不静默跑错。- 落点：DD-01 不做；SPEC §4.6 skill 格式 + 未来 DD-skill。

D3 | 运行目录与内存数据集漂移：见 A1；补一句路径纪律——每 run 独立运行目录 + prepared 快照，避免长会话内共享.dta 增量污染。

E. 外部世界入口 (S2 →设计决策；见 §1.1)

E1: web/fetch/download 受控工具 (隐私门 + retrieved_untrusted + 来源/版本 artifact)。**E2**: 复现数据获取放 approved/mixed；本地库没有的文献在线拉取可选。都在 DD-04 权限 + DD-01 artifact 链。

F. 输入模态 (S2 →设计决策；见 §1.2)

F1 vision 输入 (用户贴图/模型读自产图) 分层：ModelCapabilityProfile.vision + 投影按能力裁剪 + 图 = untrusted data + local_strict 本地视觉/OCR。**F2** 中文扫描 PDF OCR 并入同一视觉能力。**F3** mock/eval 支持 image fixture (防评测退化文本化)。

3. 修补动作清单 (按严重程度)

#	动作	落点	性质
1	ResearchSpec 加 prep (raw → prepared 脚本) + run 记 input_data_sig + “从 prepared 快照重建” 纪律	DD-01 §2.3/§3.5	S1 修
2	provenance 加 env_sig (Stata 版本 +ado+OS)	DD-01 §2.5	S1 修
3	NumericClaim.locator 升”表语义键” + 规划 esttab 表解析器 + round-trip	DD-01 §2.6 + 未来 writer/validator DD	S1 修
4	Figure evidence: figure artifact → FigureCard → writer 嵌 Word + 复现	DD-01/DD-03 + writer 模板	S2 修
5	门控严格度可配置 + 探索环 默认轻、锁只在声明主 spec 收紧	DD-02 §2/§4	S2 修
6	skill 声明 requires(ados/ stata) + preflight 检查	SPEC §4.6	S2 修
7	repro_manifest 进 L-D + author_dofile 最高风险 执行策略	DD-01 + DD-04	S2 修
8	web/fetch/download 受 控工具 + 联网复现包策略	SPEC/DD-04 权限 + 隐 私	S2 新增能力
9	vision/OCR 输入层 + 图 证据层 + ModelCapabili- tyProfile.vision	SPEC §4.2 + DD-03	S2 新增能力
10	描述/平衡表模板 + L3 评 分含表 1	writer 模板	S3 建议

评审口吻：审计不推翻架构；A/B 两类是” 现有 DD 该补的字段与算法”，C/D/E/F 是” 要新增的能力与文档”。是否把 1-3 立刻修进 DD-01（小 diff），4-6 修进 DD-02/SPEC（中 diff），8-9 作为” 能力决策” 开 SPEC 小节（新方向），7/10 随对应 DD——见开放问题。

4. 开放问题（要决策）

- 1. 联网能力：纳入哪一档隐私？（建议 approved_remote 及以上；local_strict 全禁）用途先锁” 复现包/公开数据/在线文献”，不做自由 browsing。

2. 图片：以”读自产图 + 学者贴图 + 中文扫描 OCR”为范围；deepseek 若无 vision 能力，local_strict 下视觉走本地模型——接受这个拆法吗？
3. 是否把 audit 的 S1(1-3) 直接修进 DD-01，还是先只存档本笔记等 DD-03/04 一起改？

